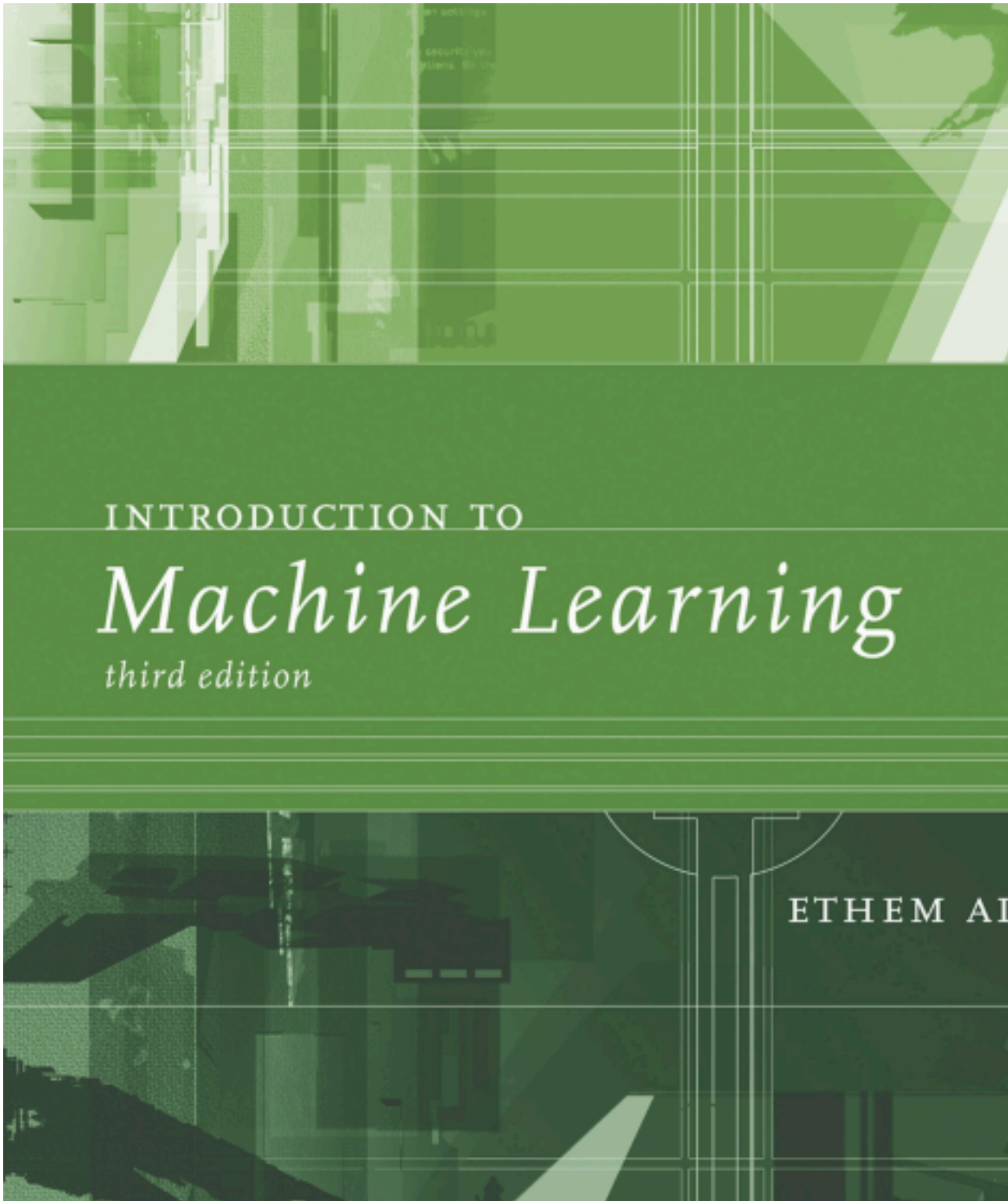




INTRODUCTION TO

Machine Learning

third edition

ETHEM ALI

Introduction
to
Machine
Learning

Third
Edition

Adaptive Computation and Machine Learning

Thomas Dietterich, Editor
Christopher Bishop, David Heckerman, Michael Jordan, and Michael
Kearns, Associate Editors

A complete list of books published in The Adaptive Computation and
Machine Learning series appears at the back of this book.

Introduction to Machine Learning

Third
Edition

Ethem Alpaydın

The MIT Press
Cambridge, Massachusetts
London, England

© 2014 Massachusetts Institute of Technology

All rights reserved. No part of this book may be reproduced in any form by any electronic or mechanical means (including photocopying, recording, or information storage and retrieval) without permission in writing from the publisher.

For information about special quantity discounts, please email
special_sales@mitpress.mit.edu.

Typeset in 10/13 Lucida Bright by the author using L^AT_EX_{2 ϵ} .
Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Information

Alpaydin, Ethem.

Introduction to machine learning / Ethem Alpaydin—3rd ed. p. cm.

Includes bibliographical references and index.

ISBN 978-0-262-02818-9 (hardcover : alk. paper)

1. Machine learning. I. Title

Q325.5.A46 2014

006.3'1—dc23 2014007214 CIP

10 9 8 7 6 5 4 3 2 1

Brief Contents

2	<i>Supervised Learning</i>	21
3	<i>Bayesian Decision Theory</i>	49
4	<i>Parametric Methods</i>	65
5	<i>Multivariate Methods</i>	93
6	<i>Dimensionality Reduction</i>	115
7	<i>Clustering</i>	161
8	<i>Nonparametric Methods</i>	185
9	<i>Decision Trees</i>	213
10	<i>Linear Discrimination</i>	239
11	<i>Multilayer Perceptrons</i>	267
12	<i>Local Models</i>	317
13	<i>Kernel Machines</i>	349
14	<i>Graphical Models</i>	387
15	<i>Hidden Markov Models</i>	417
16	<i>Bayesian Estimation</i>	445
17	<i>Combining Multiple Learners</i>	487
18	<i>Reinforcement Learning</i>	517
19	<i>Design and Analysis of Machine Learning Experiments</i>	547
	<i>Probability</i>	593

Contents

Preface xvii

Notations xxi

1 *Introduction* 1

1.1 What Is Machine Learning? 1

1.2 Examples of Machine Learning Applications 4 1.2.1
Learning Associations 4

1.2.2 Classification 5

1.2.3 Regression 9

1.2.4 Unsupervised Learning 11

1.2.5 Reinforcement Learning 13

1.3 Notes 14

1.4	Relevant Resources	17
1.5	Exercises	18
1.6	References	20
2	<i>Supervised Learning</i>	21
2.1	Learning a Class from Examples	21
2.2	Vapnik-Chervonenkis Dimension	27
2.3	Probably Approximately Correct Learning	29
2.4	Noise	30
2.5	Learning Multiple Classes	32
2.6	Regression	34
2.7	Model Selection and Generalization	37
2.8	Dimensions of a Supervised Machine Learning Algorithm	41
2.9	Notes	42

viii Contents

2.10	Exercises	43
2.11	References	47
3	<i>Bayesian Decision Theory</i>	49
3.1	Introduction	49
3.2	Classification	51
3.3	Losses and Risks	53
3.4	Discriminant Functions	55
3.5	Association Rules	56
3.6	Notes	59
3.7	Exercises	60
3.8	References	64
4	<i>Parametric Methods</i>	65
4.1	Introduction	65
4.2	Maximum Likelihood Estimation	66
4.2.1	Bernoulli Density	67
4.2.2	Multinomial Density	68
4.2.3	Gaussian (Normal) Density	68
4.3	Evaluating an Estimator: Bias and Variance	69
4.4	The Bayes' Estimator	70
4.5	Parametric Classification	73
4.6	Regression	77
4.7	Tuning Model Complexity: Bias/Variance Dilemma	80
4.8	Model Selection Procedures	83
4.9	Notes	87

4.10 Exercises	88
4.11 References	90

5 *Multivariate Methods* 93

5.1 Multivariate Data	93
5.2 Parameter Estimation	94
5.3 Estimation of Missing Values	95
5.4 Multivariate Normal Distribution	96
5.5 Multivariate Classification	100
5.6 Tuning Complexity	106
5.7 Discrete Features	108
5.8 Multivariate Regression	109
5.9 Notes	111
5.10 Exercises	112

Contents ix

5.11 References	113
-----------------	-----

6 *Dimensionality Reduction* 115

6.1 Introduction	115
6.2 Subset Selection	116
6.3 Principal Component Analysis	120
6.4 Feature Embedding	127
6.5 Factor Analysis	130
6.6 Singular Value Decomposition and Matrix Factorization	135
6.7 Multidimensional Scaling	136
6.8 Linear Discriminant Analysis	140
6.9 Canonical Correlation Analysis	145
6.10 Isomap	148
6.11 Locally Linear Embedding	150
6.12 Laplacian Eigenmaps	153
6.13 Notes	155
6.14 Exercises	157
6.15 References	158

7 *Clustering* 161

7.1 Introduction	161
7.2 Mixture Densities	162
7.3 k -Means Clustering	163
7.4 Expectation-Maximization Algorithm	167
7.5 Mixtures of Latent Variable Models	172
7.6 Supervised Learning after Clustering	173

7.7 Spectral Clustering	175
7.8 Hierarchical Clustering	176
7.9 Choosing the Number of Clusters	178
7.10 Notes	179
7.11 Exercises	180
7.12 References	182
8 <i>Nonparametric Methods</i>	185
8.1 Introduction	185
8.2 Nonparametric Density Estimation	186
8.2.1 Histogram Estimator	187
8.2.2 Kernel Estimator	188
8.2.3 k -Nearest Neighbor Estimator	190
8.3 Generalization to Multivariate Data	192
<i>x Contents</i>	
8.4 Nonparametric Classification	193
8.5 Condensed Nearest Neighbor	194
8.6 Distance-Based Classification	196
8.7 Outlier Detection	199
8.8 Nonparametric Regression: Smoothing Models	201
8.8.1 Running Mean Smoother	201
8.8.2 Kernel Smoother	203
8.8.3 Running Line Smoother	204
8.9 How to Choose the Smoothing Parameter	204
8.10 Notes	205
8.11 Exercises	208
8.12 References	210
9 <i>Decision Trees</i>	213
9.1 Introduction	213
9.2 Univariate Trees	215
9.2.1 Classification Trees	216
9.2.2 Regression Trees	220
9.3 Pruning	222
9.4 Rule Extraction from Trees	225
9.5 Learning Rules from Data	226
9.6 Multivariate Trees	230
9.7 Notes	232
9.8 Exercises	235
9.9 References	237

10	<i>Linear Discrimination</i>	239
10.1	Introduction	239
10.2	Generalizing the Linear Model	241
10.3	Geometry of the Linear Discriminant	242
10.3.1	Two Classes	242
10.3.2	Multiple Classes	244
10.4	Pairwise Separation	246
10.5	Parametric Discrimination Revisited	247
10.6	Gradient Descent	248
10.7	Logistic Discrimination	250
10.7.1	Two Classes	250
10.7.2	Multiple Classes	254
10.8	Discrimination by Regression	257

Contents xi

10.9	Learning to Rank	260
10.10	Notes	263
10.11	Exercises	263
10.12	References	266
11	<i>Multilayer Perceptrons</i>	267
11.1	Introduction	267
11.1.1	Understanding the Brain	268
11.1.2	Neural Networks as a Paradigm for Parallel Processing	269
11.2	The Perceptron	271
11.3	Training a Perceptron	274
11.4	Learning Boolean Functions	277
11.5	Multilayer Perceptrons	279
11.6	MLP as a Universal Approximator	281
11.7	Backpropagation Algorithm	283
11.7.1	Nonlinear Regression	284
11.7.2	Two-Class Discrimination	286
11.7.3	Multiclass Discrimination	288
11.7.4	Multiple Hidden Layers	290
11.8	Training Procedures	290
11.8.1	Improving Convergence	290
11.8.2	Overtraining	291
11.8.3	Structuring the Network	292
11.8.4	Hints	295

11.9 Tuning the Network Size	297
11.10 Bayesian View of Learning	300
11.11 Dimensionality Reduction	301
11.12 Learning Time	304
11.12.1 Time Delay Neural Networks	304
11.12.2 Recurrent Networks	305
11.13 Deep Learning	306
11.14 Notes	309
11.15 Exercises	311
11.16 References	313

12 *Local Models* 317

12.1 Introduction	317
12.2 Competitive Learning	318
<i>xii Contents</i>	
12.2.1 Online k -Means	318
12.2.2 Adaptive Resonance Theory	323
12.2.3 Self-Organizing Maps	324
12.3 Radial Basis Functions	326
12.4 Incorporating Rule-Based Knowledge	332
12.5 Normalized Basis Functions	333
12.6 Competitive Basis Functions	335
12.7 Learning Vector Quantization	338
12.8 The Mixture of Experts	338
12.8.1 Cooperative Experts	341
12.8.2 Competitive Experts	342
12.9 Hierarchical Mixture of Experts	342
12.10 Notes	343
12.11 Exercises	344
12.12 References	347

13 *Kernel Machines* 349

13.1 Introduction	349
13.2 Optimal Separating Hyperplane	351
13.3 The Nonseparable Case: Soft Margin Hyperplane	355
13.4 ν -SVM	358
13.5 Kernel Trick	359
13.6 Vectorial Kernels	361
13.7 Defining Kernels	364
13.8 Multiple Kernel Learning	365

13.9 Multiclass Kernel Machines	367
13.10 Kernel Machines for Regression	368
13.11 Kernel Machines for Ranking	373
13.12 One-Class Kernel Machines	374
13.13 Large Margin Nearest Neighbor Classifier	377
13.14 Kernel Dimensionality Reduction	379
13.15 Notes	380
13.16 Exercises	382
13.17 References	383
14 Graphical Models	387
14.1 Introduction	387
14.2 Canonical Cases for Conditional Independence	389
14.3 Generative Models	396

Contents xiii

14.4 d-Separation	399
14.5 Belief Propagation	399
14.5.1 Chains	400
14.5.2 Trees	402
14.5.3 Polytrees	404
14.5.4 Junction Trees	406
14.6 Undirected Graphs: Markov Random Fields	407
14.7 Learning the Structure of a Graphical Model	410
14.8 Influence Diagrams	411
14.9 Notes	412
14.10 Exercises	413
14.11 References	415
15 Hidden Markov Models	417
15.1 Introduction	417
15.2 Discrete Markov Processes	418
15.3 Hidden Markov Models	421
15.4 Three Basic Problems of HMMs	423
15.5 Evaluation Problem	423
15.6 Finding the State Sequence	427
15.7 Learning Model Parameters	429
15.8 Continuous Observations	432
15.9 The HMM as a Graphical Model	433
15.10 Model Selection in HMMs	436
15.11 Notes	438

15.12 Exercises 440

15.13 References 443

16 *Bayesian Estimation* 445

16.1 Introduction 445

16.2 Bayesian Estimation of the Parameters of a Discrete
Distribution 449

16.2.1 $K > 2$ States: Dirichlet Distribution 449

16.2.2 $K = 2$ States: Beta Distribution 450

16.3 Bayesian Estimation of the Parameters of a Gaussian
Distribution 451

16.3.1 Univariate Case: Unknown Mean, Known
Variance 451

xiv Contents

16.3.2 Univariate Case: Unknown Mean, Unknown
Variance 453

16.3.3 Multivariate Case: Unknown Mean, Unknown
Covariance 455

16.4 Bayesian Estimation of the Parameters of a Function 456

16.4.1 Regression 456

16.4.2 Regression with Prior on Noise Precision 460

16.4.3 The Use of Basis/Kernel Functions 461

16.4.4 Bayesian Classification 463

16.5 Choosing a Prior 466

16.6 Bayesian Model Comparison 467

16.7 Bayesian Estimation of a Mixture Model 470

16.8 Nonparametric Bayesian Modeling 473

16.9 Gaussian Processes 474

16.10 Dirichlet Processes and Chinese Restaurants 478

16.11 Latent Dirichlet Allocation 480

16.12 Beta Processes and Indian Buffets 482

16.13 Notes 483

16.14 Exercises 484

16.15 References 485

17 *Combining Multiple Learners* 487

17.1 Rationale 487

17.2 Generating Diverse Learners 488

17.3 Model Combination Schemes 491

17.4 Voting 492

17.5	Error-Correcting Output Codes	496
17.6	Bagging	498
17.7	Boosting	499
17.8	The Mixture of Experts Revisited	502
17.9	Stacked Generalization	504
17.10	Fine-Tuning an Ensemble	505
17.10.1	Choosing a Subset of the Ensemble	506
17.10.2	Constructing Metalearners	506
17.11	Cascading	507
17.12	Notes	509
17.13	Exercises	511
17.14	References	513

Contents xv

18 *Reinforcement Learning* 517

18.1	Introduction	517
18.2	Single State Case: K -Armed Bandit	519
18.3	Elements of Reinforcement Learning	520
18.4	Model-Based Learning	523
18.4.1	Value Iteration	523
18.4.2	Policy Iteration	524
18.5	Temporal Difference Learning	525
18.5.1	Exploration Strategies	525
18.5.2	Deterministic Rewards and Actions	526
18.5.3	Nondeterministic Rewards and Actions	527
18.5.4	Eligibility Traces	530
18.6	Generalization	531
18.7	Partially Observable States	534
18.7.1	The Setting	534
18.7.2	Example: The Tiger Problem	536
18.8	Notes	541
18.9	Exercises	542
18.10	References	544

19 *Design and Analysis of Machine Learning Experiments* 547

19.1	Introduction	547
19.2	Factors, Response, and Strategy of Experimentation	550
19.3	Response Surface Design	553
19.4	Randomization, Replication, and Blocking	554
19.5	Guidelines for Machine Learning Experiments	555
19.6	Cross-Validation and Resampling Methods	558
19.6.1	K -Fold	

	Cross-Validation	559
	19.6.2 5×2 Cross-Validation	560
	19.6.3 Bootstrapping	561
	19.7 Measuring Classifier Performance	561
	19.8 Interval Estimation	564
	19.9 Hypothesis Testing	568
	19.10 Assessing a Classification Algorithm's Performance	570
	19.10.1 Binomial Test	571
	19.10.2 Approximate Normal Test	572
	19.10.3 t Test	572
	19.11 Comparing Two Classification Algorithms	573
	19.11.1 McNemar's Test	573
	<i>xvi Contents</i>	
	19.11.2 K -Fold Cross-Validated Paired t Test	573
	19.11.3 5×2 cv Paired t Test	574
	19.11.4 5×2 cv Paired F Test	575
	19.12 Comparing Multiple Algorithms: Analysis of Variance	576
	19.13 Comparison over Multiple Datasets	580
	19.13.1 Comparing Two Algorithms	581
	19.13.2 Multiple Algorithms	583
	19.14 Multivariate Tests	584
	19.14.1 Comparing Two Algorithms	585
	19.14.2 Comparing Multiple Algorithms	586
	19.15 Notes	587
	19.16 Exercises	588
	19.17 References	590
A	Probability	593
A.1	Elements of Probability	593
A.1.1	Axioms of Probability	594
A.1.2	Conditional Probability	594
A.2	Random Variables	595
A.2.1	Probability Distribution and Density Functions	595
A.2.2	Joint Distribution and Density Functions	596
A.2.3	Conditional Distributions	596
A.2.4	Bayes' Rule	597
A.2.5	Expectation	597
A.2.6	Variance	598
A.2.7	Weak Law of Large Numbers	599
A.3	Special Random Variables	599

	A.3.1 Bernoulli Distribution	599
	A.3.2 Binomial Distribution	600
	A.3.3 Multinomial Distribution	600
	A.3.4 Uniform Distribution	600
	A.3.5 Normal (Gaussian) Distribution	601
	A.3.6 Chi-Square Distribution	602
	A.3.7 t Distribution	603
	A.3.8 F Distribution	603
	A.4 References	603
	<i>Index</i>	605

Preface

Machine learning must be one of the fastest growing fields in computer science. It is not only that the data is continuously getting “bigger,” but also the theory to process it and turn it into knowledge. In various fields of science, from astronomy to biology, but also in everyday life, as digital technology increasingly infiltrates our daily existence, as our digital footprint deepens, more data is continuously generated and collected. Whether scientific or personal, data that just lies dormant passively is not of any use, and smart people have been finding ever new ways to make use of that data and turn it into a useful product or service. In this transformation, machine learning plays a larger and larger role.

This data evolution has been continuing even stronger since the second edition appeared in 2010. Every year, datasets are getting larger. Not only has the number of observations grown, but the number of observed attributes has also increased significantly. There is more structure to the data: It is not just numbers and character strings any more but images, video, audio, documents, web pages, click logs, graphs, and so on. More and more, the data moves away from the parametric assumptions we used to make—for example, normality. Frequently, the data is dynamic and so there is a time dimension. Sometimes, our observations are multi-view—for the same object or event, we have multiple sources of information from different sensors and modalities.

Our belief is that behind all this seemingly complex and voluminous data, there lies a simple explanation. That although the data is big, it can be explained in terms of a relatively simple model with a small number of hidden factors and their interaction. Think about millions of customers who each day buy thousands of products online or from their local super market. This implies

pattern to this data. People do not shop at random. A person throwing a party buys a certain subset of products, and a person who has a baby at home buys a different subset; there are hidden factors that explain customer behavior.

This is one of the areas where significant research has been done in recent years—namely, to infer this hidden model from observed data. Most of the revisions in this new edition are related to these advances. Chapter 6 contains new sections on feature embedding, singular value decomposition and matrix factorization, canonical correlation analysis, and Laplacian eigenmaps.

There are new sections on distance estimation in chapter 8 and on kernel machines in chapter 13: Dimensionality reduction, feature extraction, and distance estimation are three names for the same devil—the ideal distance measure is defined in the space of the ideal hidden features, and they are fewer in number than the values we observe.

Chapter 16 is rewritten and significantly extended to cover such generative models. We discuss the Bayesian approach for all major machine learning models, namely, classification, regression, mixture models, and dimensionality reduction. Nonparametric Bayesian modeling, which has become increasingly popular during these last few years, is especially interesting because it allows us to adjust the complexity of the model to the complexity of data.

New sections have been added here and there, mostly to highlight different recent applications of the same or very similar methods. There is a new section on outlier detection in chapter 8. Two new sections in chapters 10 and 13 discuss ranking for linear models and kernel machines, respectively. Having added Laplacian eigenmaps to chapter 6, I also include a new section on spectral clustering in chapter 7. Given the recent resurgence of deep neural networks, it became necessary to include a new section on deep learning in chapter 11. Chapter 19 contains a new section on multivariate tests for comparison of methods.

Since the first edition, I have received many requests for the solutions to exercises from readers who use the book for self-study. In this new edition, I have included the solutions to some of the more didactic exercises. Sometimes they are complete solutions, and sometimes they give just a hint or offer only one of several possible solutions.

I would like to thank all the instructors and students who have used the previous two editions, as well as their translations into German, Chinese, and Turkish, and their reprints in India. I am always grateful to those

who send me words of appreciation, criticism, or errata, or who provide feedback in any other way. Please keep them coming. My email address is alpaydin@boun.edu.tr. The book's web site is

<http://www.cmpe.boun.edu.tr/~ethem/i2ml3e>.

It has been a pleasure to work with the MIT Press again on this third edition, and I thank Marie Lufkin Lee, Marc Lowenthal, and Kathleen Caruso for all their help and support.

Notations

x Scalar value

$\mathbf{x}$ Vector

$\mathbf{X}$ Matrix

$\mathbf{x}^T$ Transpose

$\mathbf{X}^{-1}$ Inverse

X Random variable

$P(X)$ Probability mass function when X is discrete $p(X)$ Probability density function when X is continuous $P(X|Y)$ Conditional probability of X given Y

$E[X]$ Expected value of the random variable X $\text{Var}(X)$ Variance of X

$\text{Cov}(X, Y)$ Covariance of X and Y

$\text{Corr}(X, Y)$ Correlation of X and Y

μ Mean

σ^2 Variance

Σ Covariance matrix

m Estimator to the mean

s^2 Estimator to the variance

S Estimator to the covariance matrix

$N(\mu, \sigma^2)$ Univariate normal distribution with mean μ and variance σ^2

Z Unit normal distribution: $N(0, 1)$

$N_d(\mu, \Sigma)$ d -variate normal distribution with mean vector μ and covariance matrix Σ

x Input

d Number of inputs (input dimensionality)

y Output

r Required output

K Number of outputs (classes)

N Number of training instances

z Hidden value, intrinsic dimension, latent factor

k Number of hidden dimensions, latent factors

C_i Class i

X Training sample

$\{x^t\}_{t=1}^N$ Set of x with index t ranging from 1 to N

$\{x^t, r^t\}_t$ Set of ordered pairs of input and desired output with index t

$g(x|\theta)$ Function of x defined up to a set of parameters θ

$\arg \max_{\theta} g(x|\theta)$ The argument θ for which g has its maximum value

$\arg \min_{\theta} g(x|\theta)$ The argument θ for which g has its minimum value

$E(\theta|X)$ Error function with parameters θ on the sample X

$l(\theta|X)$ Likelihood of parameters θ on the sample X

$L(\theta|X)$ Log likelihood of parameters θ on the sample X

$1(c)$ 1 if c is true, 0 otherwise

$\#\{c\}$ Number of elements for which c is true

δ_{ij} Kronecker delta: 1 if $i = j$, 0 otherwise

1

Introduction

1.1 What Is Machine Learning?

This is the age of “big data.” Once upon a time, only companies had data. There used to be computer centers where that data was stored and processed. First with the arrival of personal computers and later with the widespread use of wireless communications, we all became producers of data. Every time we buy a product, every time we rent a movie, visit a web page, write a blog, or post on the social media, even when we just walk or drive around, we are generating data.

Each of us is not only a generator but also a consumer of data. We want to have products and services specialized for us. We want our needs to be understood and interests to be predicted.

Think, for example, of a supermarket chain that is selling thousands of goods to millions of customers either at hundreds of brick-and-mortar stores all over a country or through a virtual store over the web. The details of each transaction are stored: date, customer id, goods bought and their amount, total money spent, and so forth. This typically amounts to a lot of data every day. What the supermarket chain wants is to be able to predict which customer is likely to buy which product, to maximize sales and profit. Similarly each customer wants to find the set of products best matching his/her needs.

This task is not evident. We do not know exactly which people are likely to buy this ice cream flavor or the next book of this author, see this new movie, visit this city, or click this link. Customer behavior changes in time and by geographic location. But we know that it is not completely random. People do not go to supermarkets and buy things at random. When they buy beer, they buy chips; they buy ice cream in summer and

2 1 Introduction

spices for Glühwein in winter. There are certain patterns in the data.

To solve a problem on a computer, we need an algorithm. An algorithm is a sequence of instructions that should be carried out to transform the input to output. For example, one can devise an algorithm for sorting. The input is a set of numbers and the output is their ordered list. For the same task, there may be various algorithms and we may be interested in finding the most efficient one, requiring the least number of instructions or memory or both.

For some tasks, however, we do not have an algorithm. Predicting customer behavior is one; another is to tell spam emails from legitimate ones. We

know what the input is: an email document that in the simplest case is a file of characters. We know what the output should be: a yes/no output indicating whether the message is spam or not. But we do not know how to transform the input to the output. What is considered spam changes in time and from individual to individual.

What we lack in knowledge, we make up for in data. We can easily compile thousands of example messages, some of which we know to be spam and some of which are not, and what we want is to “learn” what constitutes spam from them. In other words, we would like the computer (machine) to extract automatically the algorithm for this task. There is no need to learn to sort numbers since we already have algorithms for that, but there are many applications for which we do not have an algorithm but have lots of data.

We may not be able to identify the process completely, but we believe we can construct *a good and useful approximation*. That approximation may not explain everything, but may still be able to account for some part of the data. We believe that though identifying the complete process may not be possible, we can still detect certain patterns or regularities. This is the niche of machine learning. Such patterns may help us understand the process, or we can use those patterns to make predictions: Assuming that the future, at least the near future, will not be much different from the past when the sample data was collected, the future predictions can also be expected to be right.

Application of machine learning methods to large databases is called *data mining*. The analogy is that a large volume of earth and raw material is extracted from a mine, which when processed leads to a small amount of very precious material; similarly, in data mining, a large volume of data is processed to construct a simple model with valuable use, for example, having high predictive accuracy. Its application areas are

1.1 What Is Machine Learning? 3

abundant: In addition to retail, in finance banks analyze their past data to build models to use in credit applications, fraud detection, and the stock market. In manufacturing, learning models are used for optimization, control, and troubleshooting. In medicine, learning programs are used for medical diagnosis. In telecommunications, call patterns are analyzed for network optimization and maximizing the quality of service. In science, large amounts of data in physics, astronomy, and biology can only be analyzed fast enough by computers. The World Wide Web is huge; it is constantly growing, and searching for relevant information cannot be done manually.

But machine learning is not just a database problem; it is also a part of artificial intelligence. To be intelligent, a system that is in a changing environment should have the ability to learn. If the system can learn and adapt to such changes, the system designer need not foresee and provide solutions

for all possible situations.

Machine learning also helps us find solutions to many problems in vision, speech recognition, and robotics. Let us take the example of recognizing faces: This is a task we do effortlessly; every day we recognize family members and friends by looking at their faces or from their photographs, despite differences in pose, lighting, hair style, and so forth. But we do it unconsciously and are unable to explain how we do it. Because we are not able to explain our expertise, we cannot write the computer program. At the same time, we know that a face image is not just a random collection of pixels; a face has structure. It is symmetric. There are the eyes, the nose, the mouth, located in certain places on the face. Each person's face is a pattern composed of a particular combination of these. By analyzing sample face images of a person, a learning program captures the pattern specific to that person and then recognizes by checking for this pattern in a given image. This is one example of *pattern recognition*.

Machine learning is programming computers to optimize a performance criterion using example data or past experience. We have a model defined up to some parameters, and learning is the execution of a computer program to optimize the parameters of the model using the training data or past experience. The model may be *predictive* to make predictions in the future, or *descriptive* to gain knowledge from data, or both.

Machine learning uses the theory of statistics in building mathematical models, because the core task is making inference from a sample. The role of computer science is twofold: First, in training, we need efficient

4.1 Introduction

algorithms to solve the optimization problem, as well as to store and process the massive amount of data we generally have. Second, once a model is learned, its representation and algorithmic solution for inference needs to be efficient as well. In certain applications, the efficiency of the learning or inference algorithm, namely, its space and time complexity, may be as important as its predictive accuracy.

Let us now discuss some example applications in more detail to gain more insight into the types and uses of machine learning.

1.2 Examples of Machine Learning Applications

1.2.1 Learning Associations

In the case of retail—for example, a supermarket chain—one application of

machine learning is *basket analysis*, which is finding associations between products bought by customers: If people who buy X typically also buy Y , and if there is a customer who buys X and does not buy Y , he or she is a potential Y customer. Once we find such customers, we can target them for cross-selling.

association rule In finding an *association rule*, we are interested in learning a conditional probability of the form $P(Y|X)$ where Y is the product we would like to condition on X , which is the product or the set of products which we know that the customer has already purchased.

Let us say, going over our data, we calculate that $P(\text{chips}|\text{beer}) = 0.7$. Then, we can define the rule:

70 percent of customers who buy beer also buy chips.

We may want to make a distinction among customers and toward this, estimate $P(Y|X,D)$ where D is the set of customer attributes, for example, gender, age, marital status, and so on, assuming that we have access to this information. If this is a bookseller instead of a supermarket, products can be books or authors. In the case of a web portal, items correspond to links to web pages, and we can estimate the links a user is likely to click and use this information to download such pages in advance for faster access.

1.2 Examples of Machine Learning Applications 5

1.2.2 Classification

A credit is an amount of money loaned by a financial institution, for example, a bank, to be paid back with interest, generally in installments. It is important for the bank to be able to predict in advance the risk associated with a loan, which is the probability that the customer will default and not pay the whole amount back. This is both to make sure that the bank will make a profit and also to not inconvenience a customer with a loan over his or her financial capacity.

In *credit scoring* (Hand 1998), the bank calculates the risk given the amount of credit and the information about the customer. The information about the customer includes data we have access to and is relevant in calculating his or her financial capacity—namely, income, savings, collaterals, profession, age, past financial history, and so forth. The bank has a record of past loans containing such customer data and whether the loan was paid back or not. From this data of particular applications, the aim is to infer a general rule coding the association between a customer's attributes and his risk. That is, the machine learning system fits a model to the past data to be able to calculate the risk for a new application and then decides to accept or refuse it accordingly.

classification This is an example of a *classification* problem where there are two classes: low-risk and high-risk customers. The information about a customer makes up the *input*

to the classifier whose task is to assign the input to one of the two classes.

After training with the past data, a classification rule learned may be of the form

IF $\text{income} > \theta_1$ AND $\text{savings} > \theta_2$ THEN low-risk ELSE high-risk

for suitable values of θ_1 and θ_2 (see figure 1.1). This is an example of discriminant a *discriminant*; it is a function that separates the examples of different classes.

prediction Having a rule like this, the main application is *prediction*: Once we have a rule that fits the past data, if the future is similar to the past, then we can make correct predictions for novel instances. Given a new application with a certain income and savings, we can easily decide whether it is low risk or high-risk.

In some cases, instead of making a 0/1 (low-risk/high-risk) type decision, we may want to calculate a probability, namely, $P(Y|X)$, where X are the customer attributes and Y is 0 or 1 respectively for low-risk

6.1 Introduction

Savings

Low-risk

θ_2

High-risk

Income

θ_1

Figure 1.1 Example of a training dataset where each circle corresponds to one data instance with input values in the corresponding axes and its sign indicates the class. For simplicity, only two customer attributes, income and savings, are taken as input and the two classes are low-risk ('+') and high-risk ('-'). An example discriminant that separates the two types of examples is also shown.

and high-risk. From this perspective, we can see classification as learning an association from X to Y . Then for a given $X = x$, if we have $P(Y = 1|X = x) = 0.8$, we say that the customer has an 80 percent probability of being high-risk, or equivalently a 20 percent probability of being low-risk. We then decide whether to accept or refuse the loan depending on the possible gain and loss.

There are many applications of machine learning in *pattern recognition*.^{recognition} One is *optical character recognition*, which is recognizing character codes from their images. This is an example where there are multiple classes, as many as there are characters we would like to recognize. Especially interesting is the case when the characters are handwritten—for example, to read zip codes on envelopes or amounts on checks. People have different handwriting styles; characters may be written small or large, slanted, with a pen or pencil, and there are many possible images corresponding

1.2 Examples of Machine Learning Applications 7

to the same character. Though writing is a human invention, we do not have any system that is as accurate as a human reader. We do not have a formal description of 'A' that covers all 'A's and none of the non-'A's. Not having it, we take samples from writers and learn a definition of A-ness from these examples. But though we do not know what it is that makes an image an 'A', we are certain that all those distinct 'A's have something in common, which is what we want to extract from the examples. We know that a character image is not just a collection of random dots; it is a collection of strokes and has a regularity that we can capture by a learning program.

If we are reading a text, one factor we can make use of is the redundancy in human languages. A word is a *sequence* of characters and successive characters are not independent but are constrained by the words of the language. This has the advantage that even if we cannot recognize a character, we can still read the word. Such contextual dependencies may also occur in higher levels, between words and sentences, through the syntax and semantics of the language. There are machine learning algorithms to learn sequences and model such dependencies.

In the case of *face recognition*, the input is an image, the classes are people to be recognized, and the learning program should learn to associate the face images to identities. This problem is more difficult than optical character recognition because there are more classes, input image is larger, and a face is three-dimensional and differences in pose and lighting cause significant changes in the image. There may also be occlusion of certain inputs; for example, glasses may hide the eyes and eyebrows, and a beard may hide the chin.

In *medical diagnosis*, the inputs are the relevant information we have about

the patient and the classes are the illnesses. The inputs contain the patient's age, gender, past medical history, and current symptoms. Some tests may not have been applied to the patient, and thus these inputs would be missing. Tests take time, may be costly, and may inconvenience the patient so we do not want to apply them unless we believe that they will give us valuable information. In the case of a medical diagnosis, a wrong decision may lead to a wrong or no treatment, and in cases of doubt it is preferable that the classifier reject and defer decision to a human expert.

In *speech recognition*, the input is acoustic and the classes are words that can be uttered. This time the association to be learned is from an acoustic signal to a word of some language. Different people, because

8.1 Introduction

of differences in age, gender, or accent, pronounce the same word differently, which makes this task rather difficult. Another difference of speech is that the input is *temporal*; words are uttered in time as a sequence of speech phonemes and some words are longer than others.

Acoustic information only helps up to a certain point, and as in optical character recognition, the integration of a “language model” is critical in speech recognition, and the best way to come up with a language model is again by learning it from some large corpus of example data. The applications of machine learning to *natural language processing* is constantly increasing. Spam filtering is one where spam generators on one side and filters on the other side keep finding more and more ingenious ways to outdo each other. Summarizing large documents is another interesting example, yet another is analyzing blogs or posts on social networking sites to extract “trending” topics or to determine what to advertise. Perhaps the most impressive would be *machine translation*. After decades of research on hand-coded translation rules, it has become apparent that the most promising way is to provide a very large number of example pairs of texts in both languages and have a program figure out automatically the rules to map one to the other.

Biometrics is recognition or authentication of people using their physiological and/or behavioral characteristics that requires an integration of inputs from different modalities. Examples of physiological characteristics are images of the face, fingerprint, iris, and palm; examples of behavioral characteristics are dynamics of signature, voice, gait, and key stroke. As opposed to the usual identification procedures—photo, printed signature, or password—when there are many different (uncorrelated) inputs, forgeries (spoofing) would be more difficult and the system would be more accurate, hopefully without too much inconvenience to the users. Machine learning is used both in the separate recognizers for these different modalities and in the combination of their decisions to get an overall accept/reject decision, taking into account how reliable these different sources are.

knowledge Learning a rule from data also allows *knowledge extraction*. The rule is ^{extraction} a simple model that explains the data, and looking at this model we have an explanation about the process underlying the data. For example, once we learn the discriminant separating low-risk and high-risk customers, we have the knowledge of the properties of low-risk customers. We can then use this information to target potential low-risk customers more efficiently, for example, through advertising. Learning also performs *compression* efficiently, for example, through advertising. Learning also performs *compression*

1.2 Examples of Machine Learning Applications 9

pression in that by fitting a rule to the data, we get an explanation that is simpler than the data, requiring less memory to store and less computation to process. Once we have the rules of addition, we do not need to remember the sum of every possible pair of numbers.

OUTLIER DETECTION Another use of machine learning is outlier detection, which is finding instances that do not obey the general rule and are exceptions. The idea is that typical instances share characteristics that can be simply stated and instances that do not have those characteristics are atypical. In such a case, we are interested in finding a rule that is as simple as possible and covers as large a proportion of our typical instances as possible. Any instance that falls outside is an exception, which may be an anomaly requiring attention such as fraud; or it may be a novel, previously unseen **NOVELTY DETECTION** but valid case, and hence the other name novelty detection.

1.2.3 Regression

Let us say we want to have a system that can predict the price of a used car. Inputs are the car attributes-brand, year, engine capacity, mileage, and other information-that we believe affect a car's worth. The output is the price of the car. Such problems where the output is a number are **REGRESSION** regression problems.

Let X denote the car attributes and y be the price of the car. Again surveying the past transactions, we can collect a training data and the machine learning program fits a function to this data to learn y as a function of X . An example is given in figure 1.2 where the fitted function is of the form

$$y = WX + w_0$$

for suitable values of w and w_0 .

SUPERVISED LEARNING Both regression and classification are supervised learning problems where there is an input, X , an output, y , and the task is to learn the mapping from the

input to the output. The approach in machine learning is that we assume a model defined up to a set of parameters:

$$y = g(x; \theta)$$

where $g(\cdot)$ is the model and θ are its parameters. y is a number in regression and is a class code (e.g., 0/1) in the case of classification. $g(\cdot)$ is the regression function or in classification, it is the discriminant function separating the instances of different classes. The machine learning

10.1 Introduction

y : Price

x : Mileage

Figure 1.2 A training dataset of used cars and the function fitted. For simplicity, mileage is taken as the only input attribute and a linear model is used.

program optimizes the parameters, θ , such that the approximation error is minimized, that is, our estimates are as close as possible to the correct values given in the training set. For example in figure 1.2, the model is linear, and w_1 and w_0 are the parameters optimized for best fit to the training data. In cases where the linear model is too restrictive, we can use, for example, a quadratic

$$y = w_2 x^2 + w_1 x + w_0$$

or a higher-order polynomial, or any other nonlinear function of the input, this

time optimizing its parameters for best fit.

Another example of regression is navigation of a mobile robot, for example, an autonomous car, where the output is the angle by which the steering wheel should be turned at each time, to advance without hitting obstacles and deviating from the route. Inputs in such a case are provided by sensors on the car—for example, a video camera, GPS, and so

1.2 Examples of Machine Learning Applications 11

forth. Training data can be collected by monitoring and recording the actions of a human driver.

We can envisage other applications of regression where we are trying to optimize a function.¹ Let us say we want to build a machine that roasts coffee. The machine has many inputs that affect the quality: various settings of temperatures, times, coffee bean type, and so forth. We make a number of experiments and for different settings of these inputs, we measure the quality of the coffee, for example, as consumer satisfaction. To find the optimal setting, we fit a regression model linking these inputs to coffee quality and choose new points to sample near the optimum of the current model to look for a better configuration. We sample these points, check quality, and add these to the data and fit a new model. This is generally called *response surface design*.

Sometimes instead of estimating an absolute numeric value, we want to be able to learn relative positions. For example, in a *recommendation system* for movies, we want to generate a list ordered by how much we believe the user is likely to enjoy each. Depending on the movie attributes such as genre, actors, and so on, and using the ratings of the user he/she

ranking has already seen, we would like to be able to learn a *ranking* function that we can then use to choose among new movies.

1.2.4 Unsupervised Learning

In supervised learning, the aim is to learn a mapping from the input to an output whose correct values are provided by a supervisor. In unsupervised learning, there is no such supervisor and we only have input data. The aim is to find the regularities in the input. There is a structure to the input space such that certain patterns occur more often than others, and we want to see what generally happens and what does not. In statistics, density estimation this is called *density estimation*.

clustering One method for density estimation is *clustering* where the aim is to find clusters or groupings of input. In the case of a company with a data of past customers, the customer data contains the demographic information as well as the past transactions with the company, and the company may want to see the distribution of the profile of its customers, to see what type of customers frequently occur. In such a case, a clustering model allocates customers similar

in their attributes to the same group,

1. I would like to thank Michael Jordan for this example.

12 1 Introduction

providing the company with natural groupings of its customers; this is called *customer segmentation*. Once such groups are found, the company may decide strategies, for example, services and products, specific to different groups; this is known as *customer relationship management*. Such a grouping also allows identifying those who are outliers, namely, those who are different from other customers, which may imply a niche in the market that can be further exploited by the company.

An interesting application of clustering is in *image compression*. In this case, the input instances are image pixels represented as RGB values. A clustering program groups pixels with similar colors in the same group, and such groups correspond to the colors occurring frequently in the image. If in an image, there are only shades of a small number of colors, and if we code those belonging to the same group with one color, for example, their average, then the image is quantized. Let us say the pixels are 24 bits to represent 16 million colors, but if there are shades of only 64 main colors, for each pixel we need 6 bits instead of 24. For example, if the scene has various shades of blue in different parts of the image, and if we use the same average blue for all of them, we lose the details in the image but gain space in storage and transmission. Ideally, we would like to identify higher-level regularities by analyzing repeated image patterns, for example, texture, objects, and so forth. This allows a higher-level, simpler, and more useful description of the scene, and for example, achieves better compression than compressing at the pixel level. If we have scanned document pages, we do not have random on/off pixels but bitmap images of characters. There is structure in the data, and we make use of this redundancy by finding a shorter description of the data: 16 × 16 bitmap of 'A' takes 32 bytes; its ASCII code is only 1 byte.

In *document clustering*, the aim is to group similar documents. For example, news reports can be subdivided as those related to politics, sports, fashion, arts, and so on. Commonly, a document is represented as a *bag of words*—that is, we predefine a lexicon of N words, and each document is an N -dimensional binary vector whose element i is 1 if word i appears in the document; suffixes “-s” and “-ing” are removed to avoid duplicates and words such as “of,” “and,” and so forth, which are not informative, are not used. Documents are then grouped depending on the number of shared words. It is of course critical how the lexicon is chosen.

Machine learning methods are also used in *bioinformatics*. DNA in our genome is the “blueprint of life” and is a sequence of bases, namely, A, G,

C, and T. RNA is transcribed from DNA, and proteins are translated from the RNA. Proteins are what the living body is and does. Just as a DNA is a sequence of bases, a protein is a sequence of amino acids (as defined by bases). One application area of computer science in molecular biology is *alignment*, which is matching one sequence to another. This is a difficult string matching problem because strings may be quite long, there are many template strings to match against, and there may be deletions, insertions, and substitutions. Clustering is used in learning *motifs*, which are sequences of amino acids that occur repeatedly in proteins. Motifs are of interest because they may correspond to structural or functional elements within the sequences they characterize. The analogy is that if the amino acids are letters and proteins are sentences, motifs are like words, namely, a string of letters with a particular meaning occurring frequently in different sentences.

1.2.5 Reinforcement Learning

In some applications, the output of the system is a sequence of *actions*. In such a case, a single action is not important; what is important is the *policy* that is the sequence of correct actions to reach the goal. There is no such thing as the best action in any intermediate state; an action is good if it is part of a good policy. In such a case, the machine learning program should be able to assess the goodness of policies and learn from past good action sequences to be able to generate a policy. Such learning

reinforcement methods are called *reinforcement learning* algorithms. ^{learning} A good example is *game playing* where a single move by itself is not that important; it is the sequence of right moves that is good. A move is good if it is part of a good game playing policy. Game playing is an important research area in both artificial intelligence and machine learning. This is because games are easy to describe and at the same time, they are quite difficult to play well. A game like chess has a small number of rules but it is very complex because of the large number of possible moves at each state and the large number of moves that a game contains. Once we have good algorithms that can learn to play games well, we can also apply them to applications with more evident economic utility.

A robot navigating in an environment in search of a goal location is another application area of reinforcement learning. At any time, the robot can move in one of a number of directions. After a number of trial runs, it should learn the correct sequence of actions to reach to the goal state

14 1 Introduction

from an initial state, doing this as quickly as possible and without hitting any of the obstacles.

One factor that makes reinforcement learning harder is when the sys

tem has unreliable and partial sensory information. For example, a robot equipped with a video camera has incomplete information and thus at any time is in a *partially observable state* and should decide on its action taking into account this uncertainty; for example, it may not know its exact location in a room but only that there is a wall to its left. A task may also require a concurrent operation of *multiple agents* that should interact and cooperate to accomplish a common goal. An example is a team of robots playing soccer.

1.3 Notes

Evolution is the major force that defines our bodily shape as well as our built-in instincts and reflexes. We also learn to change our behavior during our lifetime. This helps us cope with changes in the environment that cannot be predicted by evolution. Organisms that have a short life in a well-defined environment may have all their behavior built-in, but instead of hardwiring into us all sorts of behavior for any circumstance that we could encounter in our life, evolution gave us a large brain and a mechanism to learn, such that we could update ourselves with experience and adapt to different environments. When we learn the best strategy in a certain situation, that knowledge is stored in our brain, and when the situation arises again, when we re-cognize (“cognize” means to know) the situation, we can recall the suitable strategy and act accordingly.

Learning has its limits though; there may be things that we can never learn with the limited capacity of our brains, just like we can never “learn” to grow a third arm, or an eye on the back of our head, even if either would be useful. Note that unlike in psychology, cognitive science, or neuroscience, our aim in machine learning is not to understand the processes underlying learning in humans and animals, but to build useful systems, as in any domain of engineering.

Almost all of science is fitting models to data. Scientists design experiments and make observations and collect data. They then try to extract knowledge by finding out simple models that explain the data they observed. This is called *induction* and is the process of extracting general rules from a set of particular cases.

1.3 Notes 15

We are now at a point that such analysis of data can no longer be done by people, both because the amount of data is huge and because people who can do such analysis are rare and manual analysis is costly. There is thus a growing interest in computer models that can analyze data and extract information automatically from them, that is, learn.

The methods we discuss in the coming chapters have their origins in different

scientific domains. Sometimes the same algorithm was independently invented in more than one field, following a different historical path.

In statistics, going from particular observations to general descriptions is called *inference* and learning is called *estimation*. Classification is called *discriminant analysis* in statistics (McLachlan 1992; Hastie, Tibshirani, and Friedman 2011). Before computers were cheap and abundant, statisticians could only work with small samples. Statisticians, being mathematicians, worked mostly with simple parametric models that could be analyzed mathematically. In engineering, classification is called *pattern recognition* and the approach is nonparametric and much more empirical (Duda, Hart, and Stork 2001; Webb and Copey 2011).

Machine learning is also related to *artificial intelligence* (Russell and Norvig 2009) because an intelligent system should be able to adapt to changes in its environment. Application areas like vision, speech, and robotics are also tasks that are best learned from sample data. In electrical engineering, research in *signal processing* resulted in adaptive computer vision and speech programs. Among these, the development of *hidden Markov models* for speech recognition is especially important.

In the late 1980s with advances in VLSI technology and the possibility of building parallel hardware containing thousands of processors, the field of *artificial neural networks* was reinvented as a possible theory to distribute computation over a large number of processing units (Bishop 1995). Over time, it has been realized in the neural network community that most neural network learning algorithms have their basis in statistics—for example, the multilayer perceptron is another class of nonparametric estimator—and claims of brain-like computation have started to fade.

In recent years, kernel-based algorithms, such as support vector machines, have become popular, which, through the use of kernel functions, can be adapted to various applications, especially in bioinformatics and language processing. It is common knowledge nowadays that a good representation of data is critical for learning and kernel functions turn out

16.1 Introduction

to be a very good way to introduce such expert knowledge.

Another recent approach is the use of *generative models* that explain the observed data through the interaction of a set of hidden factors. Generally, *graphical models* are used to visualize the interaction of the factors and the data, and *Bayesian formalism* allows us to define our prior information on the hidden factors and the model, as well as to infer the model parameters.

Recently, with the reduced cost of storage and connectivity, it has become possible to have very large datasets available over the Internet, and this, coupled with cheaper computation, have made it possible to run learning

algorithms on a lot of data. In the past few decades, it was generally believed that for artificial intelligence to be possible, we needed a new paradigm, a new type of thinking, a new model of computation, or a whole new set of algorithms.

Taking into account the recent successes in machine learning in various domains, it may be claimed that what we needed was not new algorithms but a lot of example data and sufficient computing power to run the algorithms on that much data. For example, the roots of support vector machines go to potential functions, linear classifiers, and neighbor-based methods, proposed in the 1950s or the 1960s; it is just that we did not have fast computers or large storage then for these algorithms to show their full potential. It may be conjectured that tasks such as machine translation, and even planning, can be solved with such relatively simple learning algorithms but trained on large amounts of example data, or through long runs of trial and error. Recent successes with “deep learning” algorithm supports this claim. Intelligence seems not to originate from some outlandish formula, but rather from the patient, almost brute-force use of a simple, straightforward algorithm.

Data mining is the name coined in the business world for the application of machine learning algorithms to large amounts of data (Witten and Frank 2011; Han and Kamber 2011). In computer science, it used to be called *knowledge discovery in databases*.

Research in these different communities (statistics, pattern recognition, neural networks, signal processing, control, artificial intelligence, and data mining) followed different paths in the past with different emphases. In this book, the aim is to incorporate these emphases together to give a unified treatment of the problems and the proposed solutions.

1.4 Relevant Resources 17

1.4 Relevant Resources

The latest research on machine learning is distributed over journals and conferences from different fields. Dedicated journals are *Machine Learning* and the *Journal of Machine Learning Research*. Journals such as *Neural Computation*, *Neural Networks*, and *IEEE Transactions on Neural Networks and Learning Systems* publish also heavily machine learning papers. Statistics journals like *Annals of Statistics* and the *Journal of the American Statistical Association* publish papers interesting from the point of view of machine learning, and many of the *IEEE Transactions* such as *Pattern Analysis and Machine Intelligence*, *Systems, Man, and Cybernetics*, *Image Processing*, and *Signal Processing* contain interesting papers related to either the theory of machine learning or one of its numerous applications.

Journals on artificial intelligence, pattern recognition, and signal processing also contain machine learning papers. Journals with an emphasis on data

mining are *Data Mining and Knowledge Discovery*, *IEEE Transactions on Knowledge and Data Engineering*, and *ACM Special Interest Group on Knowledge Discovery and Data Mining Explorations Journal*.

The major conferences on machine learning are *Neural Information Processing Systems (NIPS)*, *Uncertainty in Artificial Intelligence (UAI)*, *International Conference on Machine Learning (ICML)*, *European Conference on Machine Learning (ECML)*, *Artificial Intelligence and Statistics (AIS TATS)*, and *Computational Learning Theory (COLT)*. Conferences on pattern recognition, neural networks, artificial intelligence, fuzzy logic, and genetic algorithms, along with conferences on application areas like computer vision, speech technology, robotics, and data mining, have sessions on machine learning.

UCI Repository, at <http://archive.ics.uci.edu/ml>, contains a large number of datasets frequently used by machine learning researchers for benchmarking purposes. Another resource is the *Statlib Repository*, which is at <http://lib.stat.cmu.edu>. In addition to these, there are also repositories for particular applications, for example, computational biology, face recognition, speech recognition, and so forth.

New and larger datasets are constantly being added to these repositories. Still, some researchers believe that such repositories do not reflect the full characteristics of real data and are of limited scope, and therefore accuracies on datasets from such repositories are not indicative of anything. When some datasets from a fixed repository are used repeat-

18 1 Introduction

edly while tailoring a new algorithm, we are generating a new set of “UCI algorithms” specialized for those datasets. It is like students who are studying for a course by solving a set of example questions only. As we see in later chapters, different algorithms are better on different tasks anyway, and therefore it is best to keep one application in mind, to have one or a number of large datasets drawn for that and compare algorithms on those, for that specific task.

Most recent papers by machine learning researchers are accessible over the Internet. Most authors also make their codes and data available over the web. Videos of tutorial lectures of machine learning conferences and summer schools are mostly available too. There are also free software toolboxes and packages implementing various machine learning algorithms, and among these, Weka at <http://www.cs.waikato.ac.nz/ml/weka/>, is especially noteworthy.

1.5 Exercises

1. Imagine we have two possibilities: We can scan and email the image, or we can use

an optical character reader (OCR) and send the text file. Discuss the advantage and disadvantages of the two approaches in a comparative manner. When would one be preferable over the other?

2. Let us say we are building an OCR and for each character, we store the bitmap of that character as a template that we match with the read character pixel by pixel. Explain when such a system would fail. Why are barcode readers still used?

SOLUTION: Such a system allows only one template per character and cannot distinguish characters from multiple fonts, for example. There are standardized fonts such as OCR-A and OCR-B—the fonts we typically see on the packaging of stuff we buy—which are used with OCR software (the characters in these fonts have been slightly changed to minimize the similarities between them). Barcode readers are still used because reading barcodes is still a better (cheaper, more reliable, more available) technology than reading characters in arbitrary font, size, and styles.

3. Assume we are given the task of building a system to distinguish junk email. What is in a junk email that lets us know that it is junk? How can the computer detect junk through a syntactic analysis? What would we like the computer to do if it detects a junk email—delete it automatically, move it to a different file, or just highlight it on the screen?

SOLUTION: Typically, text-based spam filters check for the existence/absence of words and symbols. Words such as “opportunity,” “viagra,” “dollars,” and

1.5 Exercises 19

characters such as ‘\$’ and ‘!’ increase the probability that the email is spam. These probabilities are learned from a training set of example past emails that the user has previously marked as spam. We see many algorithms for this in later chapters.

The spam filters do not work with 100 percent reliability and may make errors in classification. If a junk mail is not filtered, this is not good, but it is not as bad as filtering a good mail as spam. We discuss how we can take into account the relative costs of such false positives and false negatives later on. Therefore, mail messages that the system considers as spam should not be automatically deleted but kept aside so that the user can see them if he/she wants to, especially in the early stages of using the spam filter when the system has not yet been trained sufficiently. Spam filtering is probably one of the best application areas of machine learning where learning systems can adapt to changes in the ways spam messages are generated.

4. Let us say we are given the task of building an automated taxi. Define the constraints. What are the inputs? What is the output? How can we communicate with the passenger? Do we need to communicate with the other automated taxis, that is, do we need a “language”?
5. In basket analysis, we want to find the dependence between two items X and Y . Given a database of customer transactions, how can we find these dependencies? How would we generalize this to more than two items?
6. In a daily newspaper, find five sample news reports for each category of politics, sports, and the arts. Go over these reports and find words that are used frequently

for each category, which may help you discriminate between different categories. For example, a news report on politics is likely to include words such as “government,” “recession,” “congress,” and so forth, whereas a news report on the arts may include “album,” “canvas,” or “theater.” There are also words such as “goal” that are ambiguous.

7. If a face image is a 100×100 image, written in row-major, this is a 10,000-dimensional vector. If we shift the image one pixel to the right, this will be a very different vector in the 10,000-dimensional space. How can we build face recognizers robust to such distortions?

SOLUTION: Face recognition systems typically have a preprocessing stage for normalization where the input is centered and possibly resized before recognition. This is generally done by first finding the eyes and then translating the image accordingly. There are also recognizers that do not use the face image as pixels but rather extract structural features from the image, for example, the ratio of the distance between the two eyes to the size of the whole face. Such features would be invariant to translations and size changes.

8. Take, for example, the word “machine.” Write it ten times. Also ask a friend to write it ten times. Analyzing these twenty images, try to find features,

20.1 Introduction

types of strokes, curvatures, loops, how you make the dots, and so on, that discriminate your handwriting from that of your friend’s.

9. In estimating the price of a used car, it makes more sense to estimate the percent depreciation over the original price than to estimate the absolute price. Why?

1.6 References

- Bishop, C. M. 1995. *Neural Networks for Pattern Recognition*. Oxford: Oxford University Press.
- Duda, R. O., P. E. Hart, and D. G. Stork. 2001. *Pattern Classification*, 2nd ed. New York: Wiley.
- Han, J., and M. Kamber. 2011. *Data Mining: Concepts and Techniques*, 3rd ed. San Francisco: Morgan Kaufmann.
- Hand, D. J. 1998. “Consumer Credit and Statistics.” In *Statistics in Finance*, ed. D. J. Hand and S. D. Jacka, 69–81. London: Arnold.
- Hastie, T., R. Tibshirani, and J. Friedman. 2011. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer.
- McLachlan, G. J. 1992. *Discriminant Analysis and Statistical Pattern Recognition*. New York: Wiley.
- Russell, S., and P. Norvig. 2009. *Artificial Intelligence: A Modern Approach*, 3rd ed. New York: Prentice Hall.
- Webb, A., and K. D. Copsey. 2011. *Statistical Pattern Recognition*, 3rd ed. New York:

2

Supervised Learning

We discuss supervised learning starting from the simplest case, which is learning a class from its positive and negative examples. We generalize and discuss the case of multiple classes, then regression, where the outputs are continuous.

2.1 Learning a Class from Examples

Let us say we want to learn the *class*, C , of a “family car.” We have a set of examples of cars, and we have a group of people that we survey to whom we show these cars. The people look at the cars and label them; the positive examples cars that they believe are family cars are *positive examples*, and the other negative examples cars are *negative examples*. Class learning is finding a description that is shared by all the positive examples and none of the negative examples. Doing this, we can make a prediction: Given a car that we have not seen before, by checking with the description learned, we will be able to say whether it is a family car or not. Or we can do knowledge extraction: This study may be sponsored by a car company, and the aim may be to understand what people expect from a family car.

After some discussions with experts in the field, let us say that we reach the conclusion that among all features a car may have, the features that separate a family car from other type of cars are the price and engine power. These two attributes are the *inputs* to the class recognizer. Note input that when we decide on this particular *input representation*, we are ignoring various other attributes as irrelevant. Though one may think of other attributes such as seating capacity and color that might be important for distinguishing among car types, we will consider only price and engine power to keep this example simple.

22 2 Supervised Learning x^2 : Engine power

$$x_2^t$$

$$x_1: \text{Price } x_1^t$$

Figure 2.1 Training set for the class of a “family car.” Each data point corresponds to one example car, and the coordinates of the point indicate the price and engine power of that car. ‘+’ denotes a positive example of the class (a family car), and ‘-’ denotes a negative example (not a family car); it is another type of car.

Let us denote price as the first input attribute x_1 (e.g., in U.S. dollars) and engine power as the second attribute x_2 (e.g., engine volume in cubic centimeters). Thus we represent each car using two numeric values

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (2.1)$$

and its label denotes its type

$$r = \begin{cases} 1 & \text{if } x \text{ is a positive example} \\ 0 & \text{if } x \text{ is a negative example} \end{cases} \quad (2.2)$$

$$r =$$

Each car is represented by such an ordered pair (x, r) and the training set contains N such examples

$$X = \{x^t, r^t\}_{t=1}^N \quad (2.3)$$

where t indexes different examples in the set; it does not represent time or any such order.

$$2.1 \text{ Learning a Class from Examples } 23 \quad x^2 : \text{Engine power}$$

$$e_2 C$$

$$e_1$$

x_1 : Price $p_1 p_2$

Figure 2.2 Example of a hypothesis class. The class of family car is a rectangle in the price-engine power space.

Our training data can now be plotted in the two-dimensional (x_1, x_2) space where each instance t is a data point at coordinates (x_1^t, x_2^t) and its type, namely, positive versus negative, is given by r^t (see figure 2.1).

After further discussions with the expert and the analysis of the data, we may have reason to believe that for a car to be a family car, its price and engine power should be in a certain range

$$(2.4) (p_1 \leq \text{price} \leq p_2) \text{ AND } (e_1 \leq \text{engine power} \leq e_2)$$

for suitable values of p_1, p_2, e_1 , and e_2 . Equation 2.4 thus assumes C to be a rectangle in the price-engine power space (see figure 2.2).

hypothesis class Equation 2.4 fixes H , the *hypothesis class* from which we believe C is drawn, namely, the set of rectangles. The learning algorithm then finds hypothesis the particular *hypothesis*, $h \in H$, specified by a particular quadruple of $(p_1^h, p_2^h, e_1^h, e_2^h)$, to approximate C as closely as possible.

Though the expert defines this hypothesis class, the values of the parameters are not known; that is, though we choose H , we do not know

24 2 Supervised Learning

which particular $h \in H$ is equal, or closest, to C . But once we restrict our attention to this hypothesis class, learning the class reduces to the easier problem of finding the four parameters that define h .

The aim is to find $h \in H$ that is as similar as possible to C . Let us say the hypothesis h makes a prediction for an instance x such that

1 if h classifies x as a positive example

$$0 \text{ if } h \text{ classifies } x \text{ as a negative example} \quad (2.5)$$

$$h(x) =$$

In real life we do not know $C(x)$, so we cannot evaluate how well $h(x)$ matches $C(x)$. What we have is the training set X , which is a small subset empirical error of the set of

all possible x . The *empirical error* is the proportion of training instances where *predictions* of h do not match the *required values* given in X . The error of hypothesis h given the training set X is

$$E(h|X) = \frac{1}{n} \sum_{i=1}^n 1(h(x^i) \neq r^i) \quad (2.6)$$

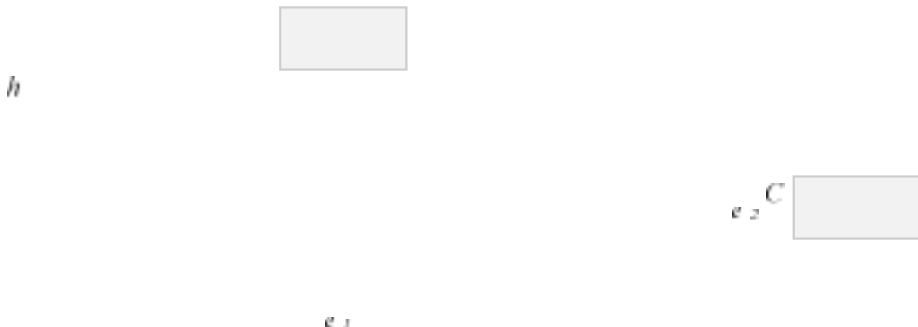
where $1(a = b)$ is 1 if $a = b$ and is 0 if $a \neq b$ (see figure 2.3).

In our example, the hypothesis class H is the set of all possible rectangles. Each quadruple $(p_1^h, p_2^h, e_1^h, e_2^h)$ defines one hypothesis, h , from H , and we need to choose the best one, or in other words, we need to find the values of these four parameters given the training set, to include all the positive examples and none of the negative examples. Note that if x_1 and x_2 are real-valued, there are infinitely many such h for which this is satisfied, namely, for which the error, E , is 0, but given a future example somewhere close to the boundary between positive and negative examples, different candidate hypotheses may make different

generalization predictions. This is the problem of *generalization*—that is, how well our hypothesis will correctly classify future examples that are not part of the training set.

One possibility is to find the *most specific hypothesis*, S , that is the tightest rectangle that includes all the positive examples and none of the negative examples (see figure 2.4). This gives us one hypothesis, $h = S$, as our induced class. Note that the actual class C may be larger than S but is most general never smaller. The *most general hypothesis*, G , is the largest rectangle we can draw that includes all the positive examples and none of the negative examples (figure 2.4). Any $h \in H$ between S and G is a valid hypothesis with no error, said to be *consistent* with the training set, and such h make version space up the *version space*. Given another training set, S , G , version space, the parameters and thus the learned hypothesis, h , can be different.

2.1 Learning a Class from Examples 25



$x \mapsto P_x$ Price

Figure 2.3 C is the actual class and h is our induced hypothesis. The point where C is 1 but h is 0 is a false negative, and the point where C is 0 but h is 1 is a false positive. Other points—namely, true positives and true negatives—are correctly classified.

Actually, depending on X and H , there may be several S_i and G_j which respectively make up the S -set and the G -set. Every member of the S -set is consistent with all the instances, and there are no consistent hypotheses that are more specific. Similarly, every member of the G -set is consistent with all the instances, and there are no consistent hypotheses that are more general. These two make up the boundary sets and any hypothesis between them is consistent and is part of the version space. There is an algorithm called candidate elimination that incrementally updates the S - and G -sets as it sees training instances one by one; see Mitchell 1997. We assume X is large enough that there is a unique S and G .

Given X , we can find S , or G , or any h from the version space and use it as our hypothesis, h . It seems intuitive to choose h halfway between S margin and G ; this is to increase the *margin*, which is the distance between the boundary and the instances closest to it (see figure 2.5). For our error function to have a minimum at h with the maximum margin, we should

26.2 Supervised Learning

x^2 : Engine power

G
C
S

x_7 : Price

Figure 2.4 S is the most specific and G is the most general hypothesis.

use an error (loss) function which not only checks whether an instance is on the correct side of the boundary but also how far away it is. That is, instead of $h(x)$ that returns 0/1, we need to have a hypothesis that returns a value which carries a measure of the distance to the boundary and we need to have a loss function which uses it, different from $1(\cdot)$ that checks for equality.

In some applications, a wrong decision may be very costly and in such a case, we can say that any instance that falls in between S and G is a doubt case of *doubt*, which we cannot label with certainty due to lack of data. In such a case, the system *rejects* the instance and defers the decision to a human expert.

Here, we assume that H includes C; that is, there exists $h \in H$, such that $E(h|X)$ is 0. Given a hypothesis class H, it may be the case that we cannot learn C; that is, there exists no $h \in H$ for which the error is 0. Thus, in any

application, we need to make sure that H is flexible enough, or has enough “capacity,” to learn C .

2.2 Vapnik-Chervonenkis Dimension 27

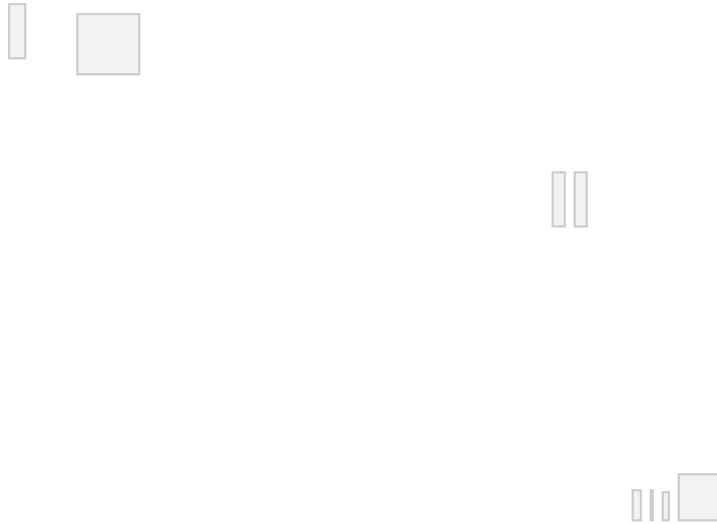


Figure 2.5 We choose the hypothesis with the largest margin, for best separation. The shaded instances are those that define (or support) the margin; other instances can be removed without affecting h .

2.2 Vapnik-Chervonenkis Dimension

Let us say we have a dataset containing N points. These N points can be labeled in 2^N ways as positive and negative. Therefore, 2^N different learning problems can be defined by N data points. If for any of these problems, we can find a hypothesis $h \in H$ that separates the positive examples from the negative, then we say H *shatters* N points. That is, any learning problem definable by N examples can be learned with no error by a hypothesis drawn from H . The maximum number of points that

VC dimension can be shattered by H is called the *Vapnik-Chervonenkis (VC) dimension* of H , is denoted as $VC(H)$, and measures the *capacity* of H .

In figure 2.6, we see that an axis-aligned rectangle can shatter four points in two dimensions. Then $VC(H)$, when H is the hypothesis class of axis-aligned

rectangles in two dimensions, is four. In calculating the VC dimension, it is enough that we find four points that can be shattered; it is not necessary that we be able to shatter *any* four points in two di-

28 2 Supervised Learning x^2

x_1

Figure 2.6 An axis-aligned rectangle can shatter four points. Only rectangles covering two points are shown.

mensions. For example, four points placed on a line cannot be shattered by rectangles. However, we cannot place five points in two dimensions *anywhere* such that a rectangle can separate the positive and negative examples for all possible labelings.

VC dimension may seem pessimistic. It tells us that using a rectangle as our hypothesis class, we can learn only datasets containing four points and not more. A learning algorithm that can learn datasets of four points is not very useful. However, this is because the VC dimension is independent of the probability distribution from which instances are drawn. In real life, the world is smoothly changing, instances close by most of the time have the same labels, and we need not worry about *all possible labelings*. There are a lot of datasets containing many more data points than four that are learnable by our hypothesis class (figure 2.1). So even hypothesis classes with small VC dimensions are applicable and are preferred over those with large VC dimensions, for example, a lookup table that has infinite VC dimension.

2.3 Probably Approximately Correct Learning

Using the tightest rectangle, S , as our hypothesis, we would like to find how many examples we need. We would like our hypothesis to be approximately correct, namely, that the error probability be bounded by some value. We also would like to be confident in our hypothesis in that we want to know that our hypothesis will be correct most of the time (if not always); so we want to be probably correct as well (by a probability we can specify).

PAC learning In *probably approximately correct* (PAC) *learning*, given a class, C , and examples drawn from some unknown but fixed probability distribution, $p(x)$, we want to find the number of examples, N , such that with probability at least $1 - \delta$, the hypothesis h has error at most ϵ , for arbitrary $\epsilon \leq 1/2$ and $\delta > 0$

$$P\{C \Delta h \leq \epsilon\} \geq 1 - \delta$$

where $C \Delta h$ is the region of difference between C and h .

In our case, because S is the tightest possible rectangle, the error region between C and $h = S$ is the sum of four rectangular strips (see figure 2.7). We would like to make sure that the probability of a positive example falling in here (and causing an error) is at most δ . For any of these strips, if we can guarantee that the probability is upper bounded by $\epsilon/4$, the error is at most $4(\epsilon/4) = \epsilon$. Note that we count the overlaps in the corners twice, and the total actual error in this case is less than $4(\epsilon/4)$. The probability that a randomly drawn example misses this strip is $1 - \epsilon/4$. The probability that all N independent draws miss the strip is $(1 - \epsilon/4)^N$, and the probability that all N independent draws miss any of the four strips is at most $4(1 - \epsilon/4)^N$, which we would like to be at most δ . We have the inequality

$$(1 - \epsilon/4)^N \leq \delta/4$$

So if we choose N and δ such that we have

$$4 \exp[-N\epsilon/4] \leq \delta$$

we can also write $4(1 - \epsilon/4)^N \leq \delta$. Dividing both sides by 4, taking (natural) log and rearranging terms, we have

$$(2.7) \quad N \geq \frac{4}{\epsilon} \log(4/\delta)$$

30 2 Supervised Learning x^2

x_1

Figure 2.7 The difference between h and C is the sum of four rectangular strips, one of which is shaded.

Therefore, provided that we take at least $(4/\epsilon) \log(4/\delta)$ independent examples from C and use the tightest rectangle as our hypothesis h , with *confidence probability* at least $1 - \delta$, a given point will be misclassified with *error probability* at most ϵ . We can have arbitrary large confidence by decreasing δ and arbitrary small error by decreasing ϵ , and we see in equation 2.7 that the number of examples is a slowly growing function of $1/\epsilon$ and $1/\delta$, linear and logarithmic, respectively.

2.4 Noise

noise *Noise* is any unwanted anomaly in the data and due to noise, the class may be more difficult to learn and zero error may be infeasible with a simple hypothesis class (see figure 2.8). There are several interpretations of noise:

There may be imprecision in recording the input attributes, which may shift the data points in the input space.

There may be errors in labeling the data points, which may relabel

h_2

h_1

x_1

Figure 2.8 When there is noise, there is not a simple boundary between the positive and negative instances, and zero misclassification error may not be possible with a simple hypothesis. A rectangle is a simple hypothesis with four parameters defining the corners. An arbitrary closed form can be drawn by piecewise functions with a larger number of control points.

positive instances as negative and vice versa. This is sometimes called *teacher noise*.

There may be additional attributes, which we have not taken into account, that affect the label of an instance. Such attributes may be *hidden* or *latent* in that they may be unobservable. The effect of these neglected attributes is thus modeled as a random component and is included in “noise.”

As can be seen in figure 2.8, when there is noise, there is not a simple boundary between the positive and negative instances and to separate them, one needs a complicated hypothesis that corresponds to a hypothesis class with larger capacity. A rectangle can be defined by four numbers, but to define a more complicated shape one needs a more complex model with a much larger number of parameters. With a complex model,

32.2 Supervised Learning

one can make a perfect fit to the data and attain zero error; see the wiggly shape in figure 2.8. Another possibility is to keep the model simple and allow some error; see the rectangle in figure 2.8.

Using the simple rectangle (unless its training error is much bigger) makes more sense because of the following:

1. It is a simple model to use. It is easy to check whether a point is inside or

outside a rectangle and we can easily check, for a future data instance, whether it is a positive or a negative instance.

2. It is a simple model to train and has fewer parameters. It is easier to find the corner values of a rectangle than the control points of an arbitrary shape. With a small training set when the training instances differ a little bit, we expect the simpler model to change less than a complex model: A simple model is thus said to have less *variance*. On the other hand, a too simple model assumes more, is more rigid, and may fail if indeed the underlying class is not that simple: A simpler model has more *bias*. Finding the optimal model corresponds to minimizing both the bias and the variance.
3. It is a simple model to explain. A rectangle simply corresponds to defining intervals on the two attributes. By learning a simple model, we can extract information from the raw data given in the training set.
4. If indeed there is mislabeling or noise in input and the actual class is really a simple model like the rectangle, then the simple rectangle, because it has less variance and is less affected by single instances, will be a better discriminator than the wiggly shape, although the simple one may make slightly more errors on the training set. Given comparable empirical error, we say that a simple (but not too simple) model would generalize better than a complex model. This principle

Occam's razor is known as *Occam's razor*, which states that *simpler explanations are more plausible* and any unnecessary complexity should be shaved off.

2.5 Learning Multiple Classes

In our example of learning a family car, we have positive examples belonging to the class family car and the negative examples belonging to all other cars. This is a *two-class* problem. In the general case, we have K

2.5 Learning Multiple Classes 33 Sports car

Engine power
?

Family car

?

Luxury sedan Price

Figure 2.9 There are three classes: family car, sports car, and luxury sedan. There are three hypotheses induced, each one covering the instances of one class and leaving outside the instances of the other two classes. '?' are reject regions where no, or more than one, class is chosen.

classes denoted as $C_i, i = 1, \dots, K$, and an input instance belongs to one and exactly one of them. The training set is now of the form

$$X = \{x^t, r^t\}_{t=1}^N$$

where r has K dimensions and

$$r_i^t = \begin{cases} 1 & \text{if } x^t \in C_i \\ 0 & \text{if } x^t \notin C_i \end{cases}$$

$$0 \text{ if } x^t \in C_j, j \neq i \quad (2.8)$$

An example is given in figure 2.9 with instances from three classes: family car, sports car, and luxury sedan.

In machine learning for classification, we would like to learn the boundary separating the instances of one class from the instances of all other classes. Thus we view a K -class classification problem as K two-class problems. The training examples belonging to C_i are the positive instances of hypothesis h_i and the examples of all other classes are the

34 2 Supervised Learning

negative instances of h_i . Thus in a K -class problem, we have K hypotheses to learn such that

$$h_i(x^t) = \begin{cases} 1 & \text{if } x^t \in C_i \\ 0 & \text{if } x^t \notin C_i \end{cases} \quad (2.9)$$

The total empirical error takes a sum over the predictions for all classes over all instances:

$$E(\{h_i\}_{i=1}^K | X) = \frac{1}{N} \sum_{t=1}^N \sum_{i=1}^K 1(h_i(x^t) \neq r_i^t) \quad (2.10)$$

For a given x , ideally only one of $h_i(x)$, $i = 1, \dots, K$ is 1 and we can choose a class. But when no, or two or more, $h_i(x)$ is 1, we cannot choose reject a class, and this is the case of *doubt* and the classifier *rejects* such cases. In our example of learning a family car, we used only one hypothesis and only modeled the positive examples. Any negative example outside is not a family car. Alternatively, sometimes we may prefer to build two hypotheses, one for the positive and the other for the negative instances. This assumes a structure also for the negative instances that can be covered by another hypothesis. Separating family cars from sports cars is such a problem; each class has a structure of its own. The advantage is that if the input is a luxury sedan, we can have both hypotheses decide negative and reject the input.

If in a dataset, we expect to have all classes with similar distribution—shapes in the input space—then the same hypothesis class can be used for all classes. For example, in a handwritten digit recognition dataset, we would expect all digits to have similar distributions. But in a medical diagnosis dataset, for example, where we have two classes for sick and healthy people, we may have completely different distributions for the two classes; there may be multiple ways for a person to be sick, reflected differently in the inputs: All healthy people are alike; each sick person is sick in his or her own way.

2.6 Regression

In classification, given an input, the output that is generated is Boolean; it is a yes/no answer. When the output is a numeric value, what we would like to learn is not a class, $C(x) \in \{0, 1\}$, but is a numeric function. In

2.6 Regression 35

machine learning, the function is not known but we have a training set of examples drawn from it

$$X = \{x^t, r^t\}_{t=1}^N$$

where r^t interpolation $\in$. If there is no noise, the task is *interpolation*. We would like to find the function $f(x)$ that passes through these points such that we have

$$r^t = f(x^t)$$

In *polynomial interpolation*, given N points, we find the $(N-1)$ st degree polynomial that we can use to predict the output for any x . This is called *extrapolation* if x is outside of the range of x^t extrapolation in the training set. In time-series prediction, for example, we have data up to the present and regression we want to predict the value for the future. In *regression*, there is noise added to the output of the unknown function

$$r^t = f(x^t(2.11)) +$$

where $f(x) \in$ is the unknown function and is random noise. The explanation for noise is that there are extra *hidden* variables that we cannot observe

$$r^t = f^*(x^t, z^t(2.12))$$

where z^t denote those hidden variables. We would like to approximate the output by our model $g(x)$. The empirical error on the training set X is

$$E(g|X) = \frac{1}{N} \sum_{t=1}^N [r^t - g(x^t)]^2(2.13)$$

Because r and $g(x)$ are numeric quantities, for example, $\in$, there is an ordering defined on their values and we can define a *distance* between values, as the square of the difference, which gives us more information than equal/not equal, as used in classification. The square of the difference is one error (loss) function that can be used; another is the absolute value of the difference. We will see other examples in the coming chapters.

Our aim is to find $g(\cdot)$ that minimizes the empirical error. Again our approach is the same; we assume a hypothesis class for $g(\cdot)$ with a small set of parameters. If we assume that $g(x)$ is linear, we have

$$g(x) = w_1x_1 + \dots + w_dx_d + w_0 = \sum_{j=1}^d w_jx_j + w_0(2.14)$$

x: Mileage

Figure 2.10 Linear, second-order, and sixth-order polynomials are fitted to the same set of points. The highest order gives a perfect fit, but given this much data it is very unlikely that the real curve is so shaped. The second order seems better than the linear fit in capturing the trend in the training data.

Let us now go back to our example in section 1.2.3 where we estimated the price of a used car. There we used a single input linear model

(2.15) $g(x) = w_1x + w_0$

where w_1 and w_0 are the parameters to learn from data. The w_1 and w_0 values should minimize

$$E(w_1, w_0 | X) = \frac{1}{N} \sum_{t=1}^N [r^t - (w_1x^t + w_0)]^2$$
 (2.16)

Its minimum point can be calculated by taking the partial derivatives of E with respect to w_1 and w_0 , setting them equal to 0, and solving for the two unknowns:

(2.17)
$$w_1 = \frac{\sum_{t=1}^N x^t r^t - \frac{1}{N} (\sum_{t=1}^N x^t)^2}{\sum_{t=1}^N x^{t2} - \frac{1}{N} (\sum_{t=1}^N x^t)^2}$$

Table 2.1 With two inputs, there are four possible cases and sixteen possible Boolean functions

x	x	h	h	h	h	h	h	h	h	h	h ₁	h ₁	h ₁	h ₁	h ₁	h ₁	h ₁
1	2	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1

1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

where $\mathbf{x} = \mathbf{x}^t/N$ and $\mathbf{r} = \mathbf{r}^t/N$. The line found is shown in figure 1.2. If the linear model is too simple, it is too constrained and incurs a large approximation error, and in such a case, the output may be taken as a higher-order function of the input—for example, quadratic

$$g(x) = w_2 x^2 + w_1 x + w_0$$

where similarly we have an analytical solution for the parameters. When the order of the polynomial is increased, the error on the training data decreases. But a high-order polynomial follows individual examples closely, instead of capturing the general trend; see the sixth-order polynomial in figure 2.10. This implies that Occam's razor also applies in the case of regression and we should be careful when fine-tuning the model complexity to match it with the complexity of the function underlying the data.

2.7 Model Selection and Generalization

Let us start with the case of learning a Boolean function from examples. In a Boolean function, all inputs and the output are binary. There are 2^d possible ways to write d binary values and therefore, with d inputs, the training set has at most 2^d examples. As shown in table 2.1, each of these can be labeled as 0 or 1, and therefore, there are 2^{2^d} possible Boolean functions of d inputs.

Each distinct training example removes half the hypotheses, namely, those whose guesses are wrong. For example, let us say we have $x_1 = 0$, $x_2 = 1$ and the output is 0; this removes $h_5, h_6, h_7, h_8, h_{13}, h_{14}, h_{15}, h_{16}$. This is one way to interpret learning: We start with all possible hypotheses and as we see more training examples, we remove those hypotheses

38.2 Supervised Learning

that are not consistent with the training data. In the case of a Boolean function, to end up with a single hypothesis we need to see *all* 2^d training examples. If the training set we are given contains only a small subset of all possible instances, as it generally does—that is, if we know what the output should be for only a small percentage of the cases—the solution is not unique. After seeing N example cases, there remain $2^{2^d - N}$ possible

ill-posed problem functions. This is an example of an *ill-posed problem* where the data by itself is not sufficient to find a unique solution.

The same problem also exists in other learning applications, in classi

fication, and in regression. As we see more training examples, we know more about the underlying function, and we carve out more hypotheses that are inconsistent from the hypothesis class, but we still are left with many consistent hypotheses.

So because learning is ill-posed, and data by itself is not sufficient to find the solution, we should make some extra assumptions to have a unique solution with the data we have. The set of assumptions we make inductive bias to have learning possible is called the *inductive bias* of the learning algorithm. One way we introduce inductive bias is when we assume a hypothesis class H . In learning the class of family cars, there are infinitely many ways of separating the positive examples from the negative examples. Assuming the shape of a rectangle is one inductive bias, and then the rectangle with the largest margin for example, is another inductive bias. In linear regression, assuming a linear function is an inductive bias, and among all lines, choosing the one that minimizes squared error is another inductive bias.

But we know that each hypothesis class has a certain capacity and can learn only certain functions. The class of functions that can be learned can be extended by using a hypothesis class with larger capacity, containing more complex hypotheses. For example, the hypothesis class that is a union of two rectangles has higher capacity, but its hypotheses are more complex. Similarly in regression, as we increase the order of the polynomial, the capacity and complexity increase. The question now is to decide where to stop.

Thus learning is not possible without inductive bias, and now the question is how to choose the right bias. This is called *model selection*, which is choosing between possible H . In answering this question, we should remember that the aim of machine learning is rarely to replicate the training data but the prediction for new cases. That is we would like to be able to generate the right output for an input instance outside the training set,

2.7 Model Selection and Generalization 39

one for which the correct output is not given in the training set. How well a model trained on the training set predicts the right output for new generalization instances is called *generalization*.

For best generalization, we should match the complexity of the hypothesis class H with the complexity of the function underlying the data. If underfitting H is less complex than the function, we have *underfitting*, for example, when trying to fit a line to data sampled from a third-order polynomial. In such a case, as we increase the complexity, the training error decreases. But if we have H that is too complex, the data is not enough to constrain it and we may end up with a bad hypothesis, $h \in H$, for example, when fitting two rectangles to data sampled from one rectangle. Or if there is noise, an overcomplex hypothesis may learn not only the underlying function but also the noise in the data and may make a bad fit, for example, when fitting a sixth-order polynomial to noisy data sampled from a third-order polynomial. This is called *overfitting*. In such a case, having more training data helps but only up

to a certain point. Given a training set and H , we can find $h \in H$ that has the minimum training error but if H is not chosen well, no matter which $h \in H$ we pick, we will not have good generalization.

triple trade-off We can summarize our discussion citing the *triple trade-off* (Dietterich 2003). In all learning algorithms that are trained from example data, there is a trade-off between three factors:

- the complexity of the hypothesis we fit to data, namely, the capacity of the hypothesis class,

- the amount of training data, and

- the generalization error on new examples.

As the amount of training data increases, the generalization error decreases. As the complexity of the model class H increases, the generalization error decreases first and then starts to increase. The generalization error of an overcomplex H can be kept in check by increasing the amount of training data but only up to a point. If the data is sampled from a line and if we are fitting a higher-order polynomial, the fit will be constrained to lie close to the line if there is training data in the vicinity; where it has not been trained, a high-order polynomial may behave erratically.

We can measure the generalization ability of a hypothesis, namely, the quality of its inductive bias, if we have access to data outside the training

40.2 Supervised Learning

set. We simulate this by dividing the dataset we have into two parts. We use one part for training (i.e., to fit a hypothesis), and the remaining validation set part is called the *validation set* and is used to test the generalization ability. That is, given a set of possible hypothesis classes H_i , for each we fit the best $h_i \in H_i$ on the training set. Then, assuming large enough training and validation sets, the hypothesis that is the most accurate on the validation set is the best one (the one that has the best inductive bias). This process is called *cross-validation*. So, for example, to find the right order in polynomial regression, given a number of candidate polynomials of different orders where polynomials of different orders correspond to H_i , for each order, we find the coefficients on the training set, calculate their errors on the validation set, and take the one that has the least validation error as the best polynomial.

Note that if we then need to report the error to give an idea about the expected error of our best model, we should not use the validation error. We have used the validation set to choose the best model, and it has effectively become a part of the training set. We need a third set, a

test set *test set*, sometimes also called the *publication set*, containing examples not used in training or validation. An analogy from our lives is when we are taking a course: the example problems that the instructor solves in class while teaching a

subject form the training set; exam questions are the validation set; and the problems we solve in our later, professional life are the test set.

We cannot keep on using the same training/validation split either, because after having been used once, the validation set effectively becomes part of training data. This will be like an instructor who uses the same exam questions every year; a smart student will figure out not to bother with the lectures and will only memorize the answers to those questions. We should always remember that the training data we use is a random sample, that is, for the same application, if we collect data once more, we will get a slightly different dataset, the fitted h will be slightly different and will have a slightly different validation error. Or if we have a fixed set which we divide for training, validation, and test, we will have different errors depending on how we do the division. These slight differences in error will allow us to estimate how large differences should be to be considered *significant* and not due to chance. That is, in choosing between two hypothesis classes H_i and H_j , we will use them both multiple times on a number of training and validation sets and check if the difference between average errors of h_i and h_j is larger than the average difference

2.8 Dimensions of a Supervised Machine Learning Algorithm 41

between multiple h_j . In chapter 19, we discuss how to design machine learning experiments using limited data to best answer our questions—for example, which is the best hypothesis class?—and how to analyze the results of these experiments so that we can achieve statistically significant conclusions minimally affected by random chance.

2.8 Dimensions of a Supervised Machine Learning Algorithm

Let us now recapitulate and generalize. We have a sample

$$X = \{x^t, r^t\}_{t=1}^N \quad (2.19)$$

The sample is *independent and identically distributed (iid)*; the ordering is not important and all instances are drawn from the same joint distribution $p(x, r)$. t

indexes one of the N instances, x^t is the arbitrary dimensional input, and r^t is the associated desired output. r^t is 0/1 for two-class learning, is a K -dimensional binary vector (where exactly one of the dimensions is 1 and all others 0) for $(K > 2)$ -class classification, and is a real value in regression.

The aim is to build a good and useful approximation to r^t using the model $g(x^t|\theta)$. In doing this, there are three decisions we must make:

1. *Model* we use in learning, denoted as

$$g(x|\theta)$$

where $g(\cdot)$ is the model, x is the input, and θ are the parameters.

$g(\cdot)$ defines the hypothesis class H , and a particular value of θ instantiates one hypothesis $h \in H$. For example, in class learning, we have taken a rectangle as our model whose four coordinates make up θ ; in linear regression, the model is the linear function of the input whose slope and intercept are the parameters learned from the data. The model (inductive bias), or H , is fixed by the machine learning system designer based on his or her knowledge of the application and the hypothesis h is chosen (parameters are tuned) by a learning algorithm using the training set, sampled from $p(x, r)$.

2. *Loss function*, $L(\cdot)$, to compute the difference between the desired output, r^t , and our approximation to it, $g(x^t|\theta)$, given the current value

42 2 Supervised Learning

of the parameters, θ . The *approximation error*, or *loss*, is the sum of losses over the individual instances

$$E(\theta|X) = \sum_t L(r^t, g(x^t|\theta)) \quad (2.20)$$

In class learning where outputs are 0/1, $L(\cdot)$ checks for equality or not; in regression, because the output is a numeric value, we have ordering information for distance and one possibility is to use the square of the difference.

3. *Optimization procedure* to find θ^* that minimizes the total error

$$\theta^* = \arg \min_{\theta} E(\theta|X) \quad (2.21)$$

where $\arg \min$ returns the argument that minimizes. In polynomial regression, we can solve analytically for the optimum, but this is not always the case. With other models and error functions, the complexity of the optimization problem becomes important. We are especially interested in whether it has a single minimum corresponding to a globally optimal solution, or whether there are multiple minima corresponding to locally optimal solutions.

For this setting to work well, the following conditions should be satisfied: First, the hypothesis class of $g(\cdot)$ should be large enough, that is, have enough capacity, to include the unknown function that generated the data that is represented in X in a noisy form. Second, there should be enough training data to allow us to pinpoint the correct (or a good enough) hypothesis from the hypothesis class. Third, we should have a good optimization method that finds the correct hypothesis given the training data.

Different machine learning algorithms differ either in the models they assume (their hypothesis class/inductive bias), the loss measures they employ, or the optimization procedure they use. We will see many examples in the coming chapters.

2.9 Notes

Mitchell proposed version spaces and the candidate elimination algorithm to incrementally build S and G as instances are given one by one;

2.10 Exercises 43

see Mitchell 1997 for a recent review. The rectangle-learning is from exercise 2.4 of Mitchell 1997. Hirsh (1990) discusses how version spaces can handle the case when instances are perturbed by small amount of noise.

In one of the earliest works on machine learning, Winston (1975) proposed the idea of a “near miss.” A near miss is a negative example that is very much like a positive example. In our terminology, we see that a near miss would be an instance that falls in the gray area between S and G , an instance which would affect the margin, and would hence be more useful for learning, than an ordinary positive or negative example. The instances that are close to the boundary are the ones that define it (or support it); those which are inside and are surrounded by many instances with the same label can be removed without affecting the boundary.

Related to this idea is *active learning* where the learning algorithm can generate instances itself and ask for them to be labeled, instead of passively being given them (Angluin 1988) (see exercise 5).

VC dimension was proposed by Vapnik and Chervonenkis in the early 1970s. A recent source is Vapnik 1995 where he writes, “Nothing is more practical than a good theory” (p. x), which is as true in machine learning as in any other branch of science. You should not rush to the computer; you can save yourself from hours of useless programming by some thinking, a notebook, and a pencil—you may also need an eraser.

The PAC model was proposed by Valiant (1984). The PAC analysis of learning a rectangle is from Blumer et al. 1989. A good textbook on com

putational learning theory covering PAC learning and VC dimension is Kearns and Vazirani 1994.

The definition of the optimization problem solved for model fitting has been getting very important in recent years. Once quite content with local descent methods that converge to the nearest good solution starting from some random initial state, nowadays we are, for example, interested in showing that the problem is convex—there is a single, global solution (Boyd and Vandenberghe 2004). As dataset sizes grow and models get more complex, we are also, for example, interested in how fast the optimization procedure converges to a solution.

2.10 Exercises

1. Let us say our hypothesis class is a circle instead of a rectangle. What are the parameters? How can the parameters of a circle hypothesis be calculated in

44 2 Supervised Learning

such a case? What if it is an ellipse? Why does it make more sense to use an ellipse instead of a circle?

SOLUTION: In the case of a circle, the parameters are the center and the radius (see figure 2.11). We then need to find S and G where S is the tightest circle that includes all the positive examples and G is the largest circle that includes all the positive examples and no negative example; any circle between them is a consistent hypothesis.

It makes more sense to use an ellipse because the two axes need not have the same scale and an ellipse has two separate parameters for the widths in the two axes rather than a single radius. Actually, price and engine power are positively correlated; the price of a car tends to increase as its engine power increases, and hence it makes more sense to use an oblique ellipse—we will see such models in chapter 5.

x_2^2 : Engine

power C_2

r

c
 c

c_1
 C

x_1 : Price

Figure 2.11 Hypothesis class is a circle with two parameters, the coordinates of its center and its radius.

2. Imagine our hypothesis is not one rectangle but a union of two (or $m > 1$) rectangles. What is the advantage of such a hypothesis class? Show that any class can be represented by such a hypothesis class with large enough m . SOLUTION: In the case when there is a single rectangle, all the positive instances should form one single group; with two rectangles, for example (see figure 2.12), the positive instances can form two, possibly disjoint clusters in the input space. Note that each rectangle corresponds to a conjunction on the two input attributes, and having multiple rectangles corresponds to a disjunction. Any logical formula can be written as a disjunction of conjunctions.

2.10 Exercises 45

In the worst case ($m = N$), we have a separate rectangle for each positive instance.

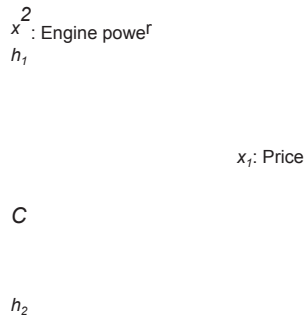


Figure 2.12 Hypothesis class is a union of two rectangles.

3. In many applications, wrong decisions—namely, false positives and false negatives—have a monetary cost, and these two costs may be different. What is the relationship between the positioning of h between S and G and the relative costs of these?

SOLUTION: We can see that S makes no false positives, but only false negatives; similarly, G makes no false negatives, only false positives. So if false positives and false negatives are equally bad, we want our h to be halfway; if false positives are costlier, we want h to be closer to S ; if false negatives are costlier, h should be closer to G .

4. The complexity of most learning algorithms is a function of the training set. Can you propose a filtering algorithm that finds redundant instances? SOLUTION: The instances that affect the hypothesis are those that are in the vicinity of instances with a different label. A positive instance that is surrounded on all sides by many positive instances is not needed, nor is a negative instance surrounded by many negative instances. We discuss such *neighbor-based* methods in chapter 8.

5. If we have a supervisor who can provide us with the label for any x , where should we choose x to learn with fewer queries?

SOLUTION: The region of ambiguity is between S and G . It would be best to be given queries there, so that we can make this region of doubt smaller. If a

46 2 Supervised Learning

given instance there turns out to be positive, this means we can make S larger up to that instance; if it is negative, this means we can shrink G down until there.

6. In equation 2.13, we summed up the squares of the differences between the actual value and the estimated value. This error function is the one most frequently used, but it is one of several possible error functions. Because it sums up the squares of the differences, it is not robust to outliers. What would be a better error function to implement *robust regression*?
7. Derive equation 2.17.
8. Assume our hypothesis class is the set of lines, and we use a line to separate the positive and negative examples, instead of bounding the positive examples as in a rectangle, leaving the negatives outside (see figure 2.13). Show that the VC dimension of a line is 3.

x^2

x_1

Figure 2.13 A line separating positive and negative instances.

9. Show that the VC dimension of the triangle hypothesis class is 7 in two dimensions. (Hint: For best separation, it is best to place the seven points equidistant on a circle.)
10. Assume as in exercise 8 that our hypothesis class is the set of lines. Write down an error function that not only minimizes the number of misclassifications but also maximizes the margin.
11. One source of noise is error in the labels. Can you propose a method to find data points that are highly likely to be mislabeled?

2.11 References

- Angluin, D. 1988. "Queries and Concept Learning." *Machine Learning* 2:319–342.
- Blumer, A., A. Ehrenfeucht, D. Haussler, and M. K. Warmuth. 1989. "Learnability and the Vapnik-Chervonenkis Dimension." *Journal of the ACM* 36:929–965.
- Boyd, S., and L. Vandenberg. 2004. *Convex Optimization*. Cambridge, UK: Cambridge University Press.
- Dietterich, T. G. 2003. "Machine Learning." In *Nature Encyclopedia of Cognitive Science*. London: Macmillan.
- Hirsh, H. 1990. *Incremental Version Space Merging: A General Framework for Concept Learning*. Boston: Kluwer.
- Kearns, M. J., and U. V. Vazirani. 1994. *An Introduction to Computational Learning Theory*. Cambridge, MA: MIT Press.
- Mitchell, T. 1997. *Machine Learning*. New York: McGraw-Hill.
- Valiant, L. 1984. "A Theory of the Learnable." *Communications of the ACM* 27:1134–1142.
- Vapnik, V. N. 1995. *The Nature of Statistical Learning Theory*. New York: Springer.
- Winston, P. H. 1975. "Learning Structural Descriptions from Examples." In *The Psychology of Computer Vision*, ed. P. H. Winston, 157–209. New York: McGraw-Hill.

3 *Bayesian Decision Theory*

We discuss probability theory as the framework for making decisions under uncertainty. In classification, Bayes' rule is used to calculate the probabilities of the classes. We generalize to discuss how we can make rational decisions among multiple actions to minimize expected risk. We also discuss learning association rules from data.

3.1 Introduction

Programming computers to make inference from data is a cross between statistics and computer science, where statisticians provide the mathematical

framework of making inference from data and computer scientists work on the efficient implementation of the inference methods.

Data comes from a process that is not completely known. This lack of knowledge is indicated by modeling the process as a random process. Maybe the process is actually deterministic, but because we do not have access to complete knowledge about it, we model it as random and use probability theory to analyze it. At this point, it may be a good idea to jump to the appendix and review basic probability theory before continuing with this chapter.

Tossing a coin is a random process because we cannot predict at any toss whether the outcome will be heads or tails—that is why we toss coins, or buy lottery tickets, or get insurance. We can only talk about the probability that the outcome of the next toss will be heads or tails. It may be argued that if we have access to extra knowledge such as the exact composition of the coin, its initial position, the force and its direction that is applied to the coin when tossing it, where and how it is caught, and so forth, the exact outcome of the toss can be predicted.

50 3 Bayesian Decision Theory

The extra pieces of knowledge that we do not have access to are named unobservable the *unobservable variables*. In the coin tossing example, the only *observable variable* is the outcome of the toss. Denoting the unobservables by z and the observable as x , in reality we have

$$x = f(z)$$

where $f(\cdot)$ is the deterministic function that defines the outcome from the unobservable pieces of knowledge. Because we cannot model the process this way, we define the outcome X as a random variable drawn from a probability distribution $P(X = x)$ that specifies the process.

The outcome of tossing a coin is heads or tails, and we define a random variable that takes one of two values. Let us say $X = 1$ denotes that the outcome of a toss is heads and $X = 0$ denotes tails. Such X are Bernoulli distributed where the parameter of the distribution p_o is the probability that the outcome is heads:

$$P(X = 1) = p_o \text{ and } P(X = 0) = 1 - P(X = 1) = 1 - p_o$$

Assume that we are asked to predict the outcome of the next toss. If we know p_o , our prediction will be heads if $p_o > 0.5$ and tails otherwise. This is because if we choose the more probable case, the probability of error, which is 1 minus the probability of our choice, will be minimum. If this is a fair coin with $p_o = 0.5$, we have no better means of prediction than choosing heads all the time or tossing a fair coin ourselves!

If we do not know $P(X)$ and want to estimate this from a given sample,

sample then we are in the realm of statistics. We have a *sample*, X , containing examples drawn from the probability distribution of the observables x^t , denoted as $p(x)$. The aim is to build an approximator to it, $\hat{p}(x)$, using the sample X .

In the coin tossing example, the sample contains the outcomes of the past N tosses. Then using X , we can estimate p_o , which is the parameter that uniquely specifies the distribution. Our estimate of p_o is

$$\hat{p}_o = \frac{\#\{\text{tosses with outcome heads}\}}{\#\{\text{tosses}\}}$$

Numerically using the random variables, x^t is 1 if the outcome of toss t is heads and 0 otherwise. Given the sample {heads, heads, heads, tails, heads, tails, tails, heads, heads}, we have $X = \{1, 1, 1, 0, 1, 0, 0, 1, 1\}$ and the estimate is

$$\hat{p}_o = \frac{\sum_{t=1}^N x^t}{N} = \frac{6}{9}$$

3.2 Classification 51

3.2 Classification

We discussed credit scoring in section 1.2.2, where we saw that in a bank, according to their past transactions, some customers are low-risk in that they paid back their loans and the bank profited from them and other customers are high-risk in that they defaulted. Analyzing this data, we would like to learn the class “high-risk customer” so that in the future, when there is a new application for a loan, we can check whether that person obeys the class description or not and thus accept or reject the application. Using our knowledge of the application, let us say that we decide that there are two pieces of information that are observable. We observe them because we have reason to believe that they give us an idea about the credibility of a customer. Let us say, for example, we observe customer’s yearly income and savings, which we represent by two random variables X_1 and X_2 .

It may again be claimed that if we had access to other pieces of knowledge such as the state of economy in full detail and full knowledge about the customer, his or her intention, moral codes, and so forth, whether someone is a low-risk or high-risk customer could have been deterministically calculated. But these are nonobservables and with what we can observe, the credibility of a customer is denoted by a Bernoulli random variable C conditioned on the observables $X = [X_1, X_2]^T$ where $C = 1$ indicates a high-risk customer and $C = 0$ indicates a low-risk customer. Thus if we know $P(C|X_1, X_2)$, when a new application arrives with $X_1 = x_1$ and $X_2 = x_2$, we can

choose

$1|x_1, x_2) > 0.5$ $C = 0$
otherwise

$C = 1$ if $P(C =$

or equivalently

$C = 1$ if $P(C = 1|x_1, x_2) > P(C = 0|x_1, x_2)$

$C = 0$ otherwise (3.1)
choose

The probability of error is $1 - \max(P(C = 1|x_1, x_2), P(C = 0|x_1, x_2))$. This example is similar to the coin tossing example except that here, the Bernoulli random variable C is conditioned on two other observable variables. Let us denote by x the vector of observed variables, $x = [x_1, x_2]^T$.

Bayes' rule The problem then is to be able to calculate $P(C|x)$. Using *Bayes' rule*, it can be written as

$$P(C|x) = P(C)p(x|C)$$

$p(x)$ (3.2)

52 3 Bayesian Decision Theory

prior probability $P(C = 1)$ is called the *prior probability* that C takes the value 1, which in our example corresponds to the probability that a customer is high risk, regardless of the x value—It is the proportion of high-risk customers in our customer base. It is called the prior probability because it is the knowledge we have as to the value of C *before* looking at the observables x , satisfying

$$P(C = 0) + P(C = 1) = 1$$

class likelihood $p(x|C)$ is called the *class likelihood* and is the conditional probability that an event belonging to C has the associated observation value x . In our case, $p(x_1, x_2|C = 1)$ is the probability that a high-risk customer has his or her $X_1 = x_1$ and $X_2 = x_2$. It is what the data tells us regarding the class.

evidence $p(x)$, the *evidence*, is the marginal probability that an observation x is seen, regardless of whether it is a positive or negative example.

$$p(x) = \sum_C p(x, C) = p(x|C = 1)P(C = 1) + p(x|C = 0)P(C = 0) \quad (3.3)$$

Combining the prior and what the data tells us using Bayes' rule, we calculate the *posterior probability* of the concept, $P(C|x)$, *after* having ^{probability} seen the observation, x .

posterior = prior $\times$ likelihood

Because of normalization by the evidence, the posteriors sum up to 1:

$$P(C=0|x) + P(C=1|x) = 1$$

Once we have the posteriors, we decide by using equation 3.1. For now, we assume that we know the prior and likelihoods; in later chapters, we discuss how to estimate $P(C)$ and $p(x|C)$ from a given training sample. In the general case, we have K mutually exclusive and exhaustive classes; $C_i, i = 1, \dots, K$; for example, in optical digit recognition, the input is a bitmap image and there are ten classes. We have the prior probabilities satisfying

$$(3.4) \quad P(C_i) \geq 0 \text{ and } \sum_{i=1}^K P(C_i) = 1$$

$p(x|C_i)$ is the probability of seeing x as the input when it is known to
3.3 Losses and Risks 53

belong to class C_i . The posterior probability of class C_i can be calculated as

$$P(C_i|x) = p(x|C_i)P(C_i) / \sum_{k=1}^K p(x|C_k)P(C_k) \quad (3.5)$$

Bayes' classifier and for minimum error, the *Bayes' classifier* chooses the class with the highest posterior probability; that is, we

$$(3.6) \quad \text{choose } C_i \text{ if } P(C_i|x) = \max_k P(C_k|x)$$

3.3 Losses and Risks

It may be the case that decisions are not equally good or costly. A financial institution when making a decision for a loan applicant should take into account the potential gain and loss as well. An accepted low-risk applicant increases profit, while a rejected high-risk applicant decreases loss. The loss for a high-risk applicant erroneously accepted may be different from the potential gain for an erroneously rejected low-risk applicant. The situation is much more critical and far from symmetry in other domains like medical diagnosis or earthquake prediction.

Let us define action α_i as the decision to assign the input to class C_i loss function and λ_{ik} as the loss incurred for taking action α_i when the input actually expected risk belongs to C_k . Then the *expected risk* for taking action α_i is $R(\alpha_i|x) = \sum_{k=1}^K \lambda_{ik} P(C_k|x)$

$$(3.7) \sum_{k=1}^K \lambda_{ik} P(C_k|x)$$

and we choose the action with minimum risk:

Choose α_i if $R(\alpha_i|x) = \min_k R(\alpha_k|x)$

$$(3.8) R(\alpha_k|x) = \sum_{i=1}^K \lambda_{ik} P(C_i|x)$$

Let us define K actions $\alpha_i, i = 1, \dots, K$, where α_i is the action of assigning x to C_i 0/1 loss. In the special case of the 0/1 loss where

$$\lambda_{ik} = \begin{cases} 0 & \text{if } i = k \\ 1 & \text{if } i \neq k \end{cases} \quad (3.9)$$

$$\lambda_{ik} = \begin{cases} 0 & \text{if } i = k \\ 1 & \text{if } i \neq k \end{cases}$$

all correct decisions have no loss and all errors are equally costly. The risk of taking action α_i is

$$\begin{aligned} R(\alpha_i|x) &= \sum_{k=1}^K \lambda_{ik} P(C_k|x) \\ &= \sum_{k=1}^K P(C_k|x) - P(C_i|x) \\ &= 1 - P(C_i|x) \end{aligned}$$

because $\sum_{k=1}^K P(C_k|x) = 1$. Thus to minimize risk, we choose the most probable class. In later chapters, for simplicity, we will always assume this case and choose the class with the highest posterior, but note that this is indeed a special case and rarely do applications have a symmetric, 0/1 loss. In the general case, it is a simple postprocessing to go from posteriors to risks and to take the action to minimize the risk.

In some applications, wrong decisions—namely, misclassifications—may have very high cost, and it is generally required that a more complex—for example, manual—decision is made if the automatic system has low certainty of its decision. For example, if we are using an optical digit recognizer to read postal codes on envelopes, wrongly recognizing the code causes the envelope to be sent to a wrong destination.

In such a case, we define an additional action of *reject* or *doubt*, α_{K+1} , with $\alpha_i, i = 1, \dots, K$, being the usual actions of deciding on classes $C_i, i = 1, \dots, K$ (Duda, Hart, and Stork 2001).

A possible loss function is

$$(3.10) \quad \lambda_{ik} = \begin{cases} 1 & \text{if } i = k \\ \lambda & \text{if } i \neq k \\ 0 & \text{otherwise} \end{cases}$$

where $0 < \lambda < 1$ is the loss incurred for choosing the $(K + 1)$ st action of reject. Then the risk of reject is

$$(3.11) \quad R(\alpha_{K+1}|x) = \sum_{k=1}^K \lambda P(C_k|x)$$

and the risk of choosing class C_i is

$$(3.12) \quad R(\alpha_i|x) = \sum_{k=i}^K P(C_k|x)$$

The optimal decision rule is to

choose C_i if $R(\alpha_i|x) < R(\alpha_k|x)$ for all $k \neq i$ and

$$(3.13) \quad \text{reject if } R(\alpha_{K+1}|x) < R(\alpha_i|x), i = 1, \dots, K$$

3.4 Discriminant Functions 55

Given the loss function of equation 3.10, this simplifies to

choose C_i if $P(C_i|x) > P(C_k|x)$ for all $k \neq i$ and

$$(3.14) \quad \text{reject otherwise}$$

This whole approach is meaningful if $0 < \lambda < 1$: If $\lambda = 0$, we always reject; a reject is as good as a correct classification. If $\lambda \geq 1$, we never reject; a reject is as costly as, or costlier than, an error.

In the case of reject, we are choosing between the automatic decision made by the computer program and human decision that is costlier but assumed to have a higher probability of being correct. Similarly, we can imagine a cascade of multiple automatic decision makers, which as we proceed are costlier but have a higher chance of being correct; we discuss such cascades in chapter 17 where we talk about combining multiple learners.

3.4 Discriminant Functions

discriminant Classification can also be seen as implementing a set of *discriminant functions*, $g_i(x)$, $i = 1, \dots, K$, such that we

$$(3.15) \quad \text{choose } C_i \text{ if } g_i(x) = \max_k g_k(x)$$

We can represent the Bayes' classifier in this way by setting

$$g_i(x) = -R(\alpha_i|x)$$

and the maximum discriminant function corresponds to minimum conditional risk. When we use the 0/1 loss function, we have

$$g_i(x) = P(C_i|x)$$

or ignoring the common normalizing term, $p(x)$, we can write

$$g_i(x) = p(x|C_i)P(C_i)$$

decision regions This divides the feature space into K *decision regions* $R_1, \dots, R_K$, where $R_i = \{x | g_i(x) = \max_k g_k(x)\}$. The regions are separated by *decision boundaries*, surfaces in feature space where ties occur among the largest discriminant functions (see figure 3.1).

56 3 Bayesian Decision Theory

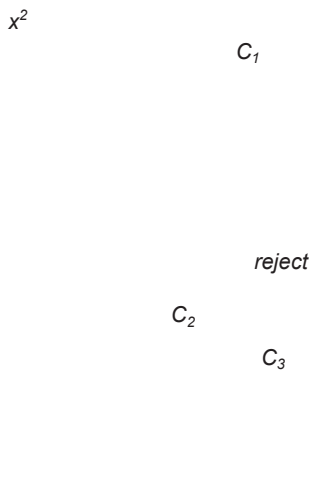


Figure 3.1 Example of decision regions and decision boundaries.

When there are two classes, we can define a single discriminant $g(x)$

$$= g_1(x) - g_2(x)$$

and we

C_2 otherwise

choose

$$C_1 \text{ if } g(x) > 0$$

An example is a two-class learning problem where the positive examples can be taken as C_1 and the negative examples as C_2 . When $K = 2$, dichotomizer the classification system is a *dichotomizer* and for $K \geq 3$, it is a *poly* ^{polychotomizer} *chotomizer*.

3.5 Association Rules

association rule An *association rule* is an implication of the form $X \rightarrow Y$ where X is the *antecedent* and Y is the *consequent* of the rule. One example of association rules is in *basket analysis* where we want to find the dependency between two items X and Y . The typical application is in retail where X and Y are items sold, as we discussed in section 1.2.1.

3.5 Association Rules 57

In learning association rules, there are three measures that are frequently calculated:

support *Support* of the association rule $X \rightarrow Y$:

$$\text{Support}(X, Y) \equiv P(X, Y) = \frac{\#\{\text{customers who bought } X \text{ and } Y\}}{\#\{\text{customers}\}} \quad (3.16)$$

confidence *Confidence* of the association rule $X \rightarrow Y$:

$$\begin{aligned} \text{Confidence}(X \rightarrow Y) &\equiv P(Y|X) = \frac{P(X, Y)}{P(X)} \\ &= \frac{\#\{\text{customers who bought } X \text{ and } Y\}}{\#\{\text{customers who bought } X\}} \quad (3.17) \end{aligned}$$

lift *Lift*, also known as *interest* of the association rule $X \rightarrow Y$: interest

$$P(X)P(Y) = P(Y|X)$$

$$\text{Lift}(X \rightarrow Y) = \frac{P(X, Y)}{P(X)P(Y)}$$

$$P(Y) \quad (3.18)$$

There are other measures as well (Omiecinski 2003), but these three, especially the first two, are the most widely known and used. *Confidence* is the conditional probability, $P(Y|X)$, which is what we normally calculate. To be able to say that the rule holds with enough confidence, this value should be close to 1 and significantly larger than $P(Y)$, the overall probability of people buying Y . We are also interested in maximizing the *support* of the rule, because even if there is a dependency with a strong confidence value, if the number of such customers is small, the rule is worthless. Support shows the statistical significance of the rule, whereas confidence shows the strength of the rule. The minimum support and confidence values are set by the company, and all rules with higher support and confidence are searched for in the database.

If X and Y are independent, then we expect lift to be close to 1; if the ratio differs—if $P(Y|X)$ and $P(Y)$ are different—we expect there to be a dependency between the two items: If the lift is more than 1, we can say that X makes Y more likely, and if the lift is less than 1, having X makes Y less likely.

These formulas can easily be generalized to more than two items. For example, $\{X, Y, Z\}$ is a three-item set, and we may look for a rule, such as $X, Z \rightarrow Y$, that is, $P(Y|X, Z)$. We are interested in finding all such rules having high enough support and confidence and because a sales database